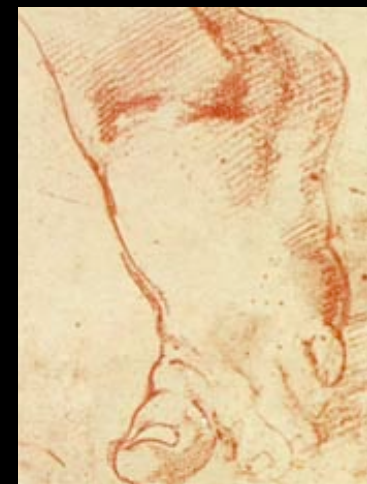




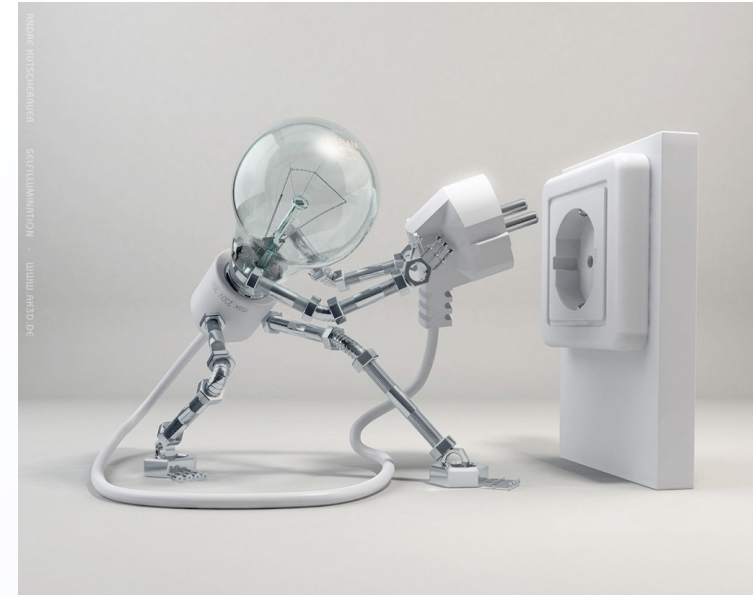
Shading



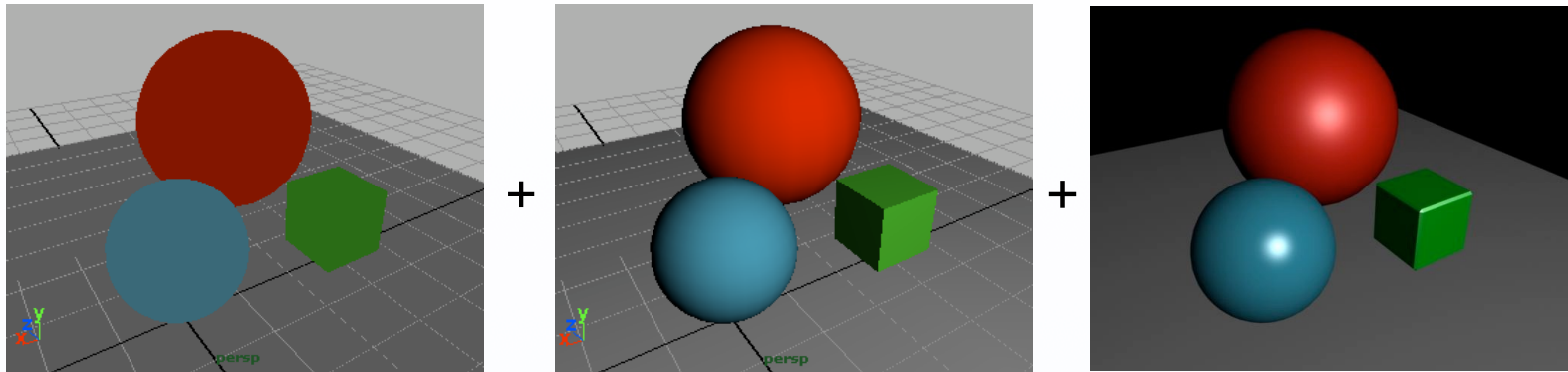
Michelangelo 1510

Bisher

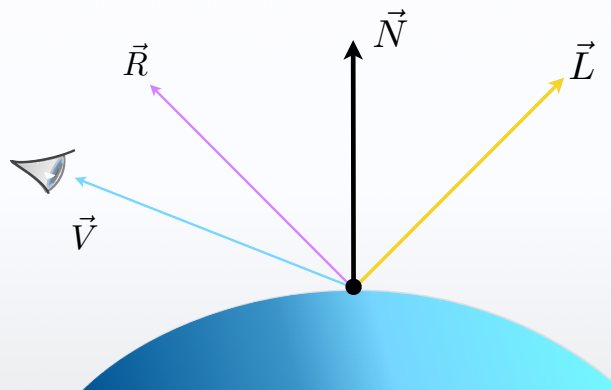
- Beleuchtung an einem Punkt berechnet
- Local Illumination
 - Lambert, Blinn, Phong, Cook/Torrance
 - Darstellung diverser Oberflächen
 - Wechselwirkung nur mit primären Lichtquellen
 - keine Wechselwirkung zwischen den Objekten
- Global Illumination
 - Interaktionen zwischen den Objekten
 - Raytracing (Reflexion, Refraktion)
 - Radiosity (Diffuse)
 - Photon Mapping
- Für eine schnelle Darstellung brauchen wir eine vereinfachte Darstellungsweise
 - Nicht jeden Pixel berechnen
 - Elterpckwerte berechnen und dazwischen interpolieren,...



Beleuchtungsmodell an einem Punkt

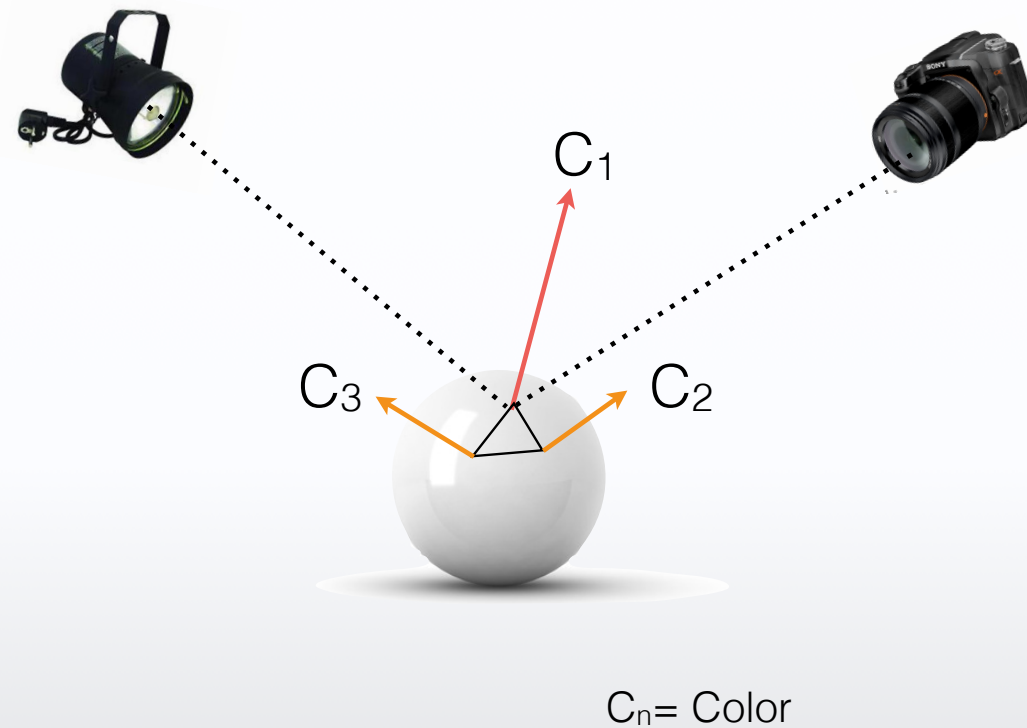


$$i_{ges} = i_a + i_d + i_s$$



$$I = I_a \cdot k_a + \sum_{p=0}^m I_p \cdot [k_d(\overline{N} \cdot \overline{L}) + k_s(\overline{V} \cdot \overline{R})^n]$$

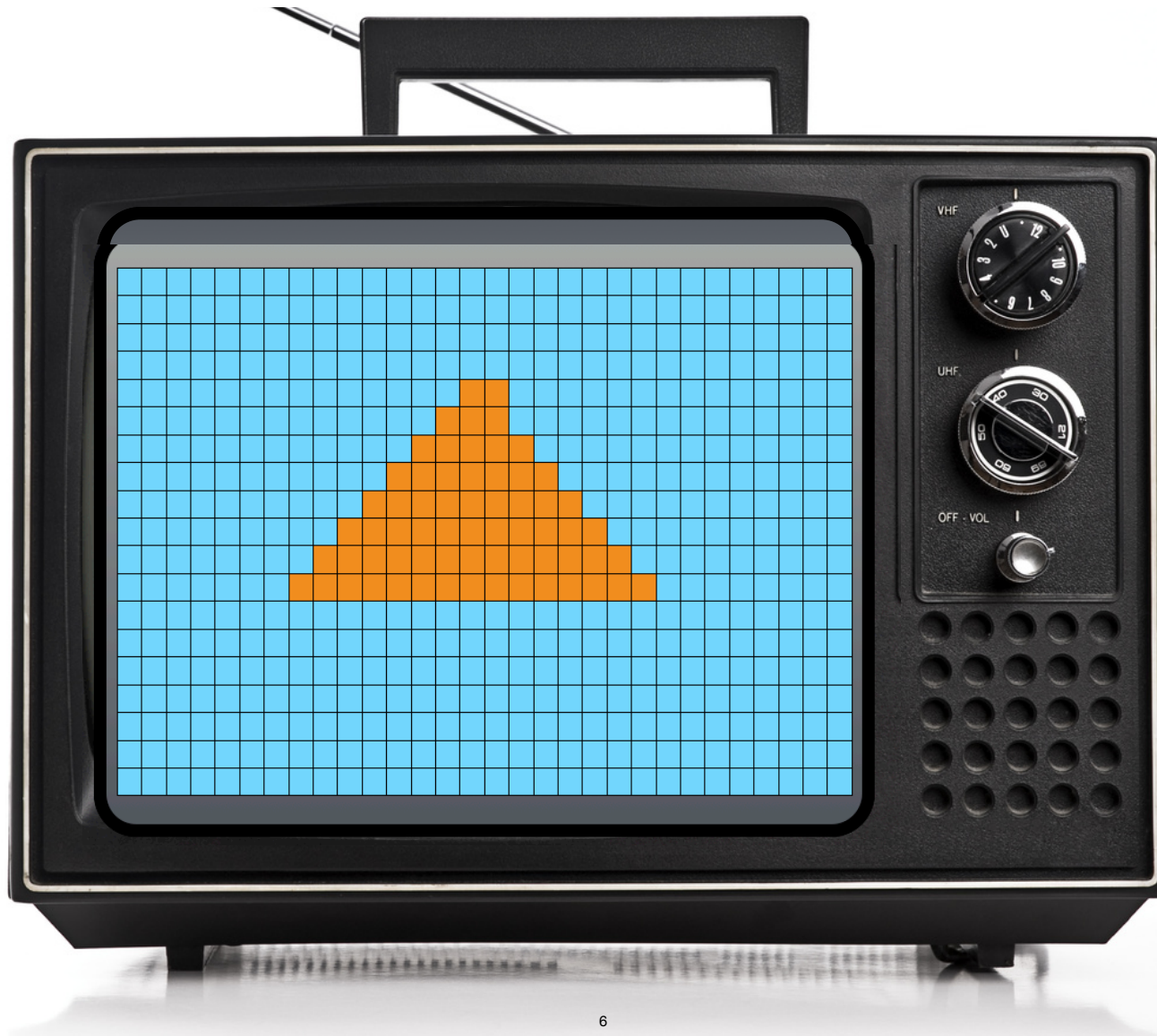
Ein Polygon: Drei Eckwerte, Drei Farbwerte



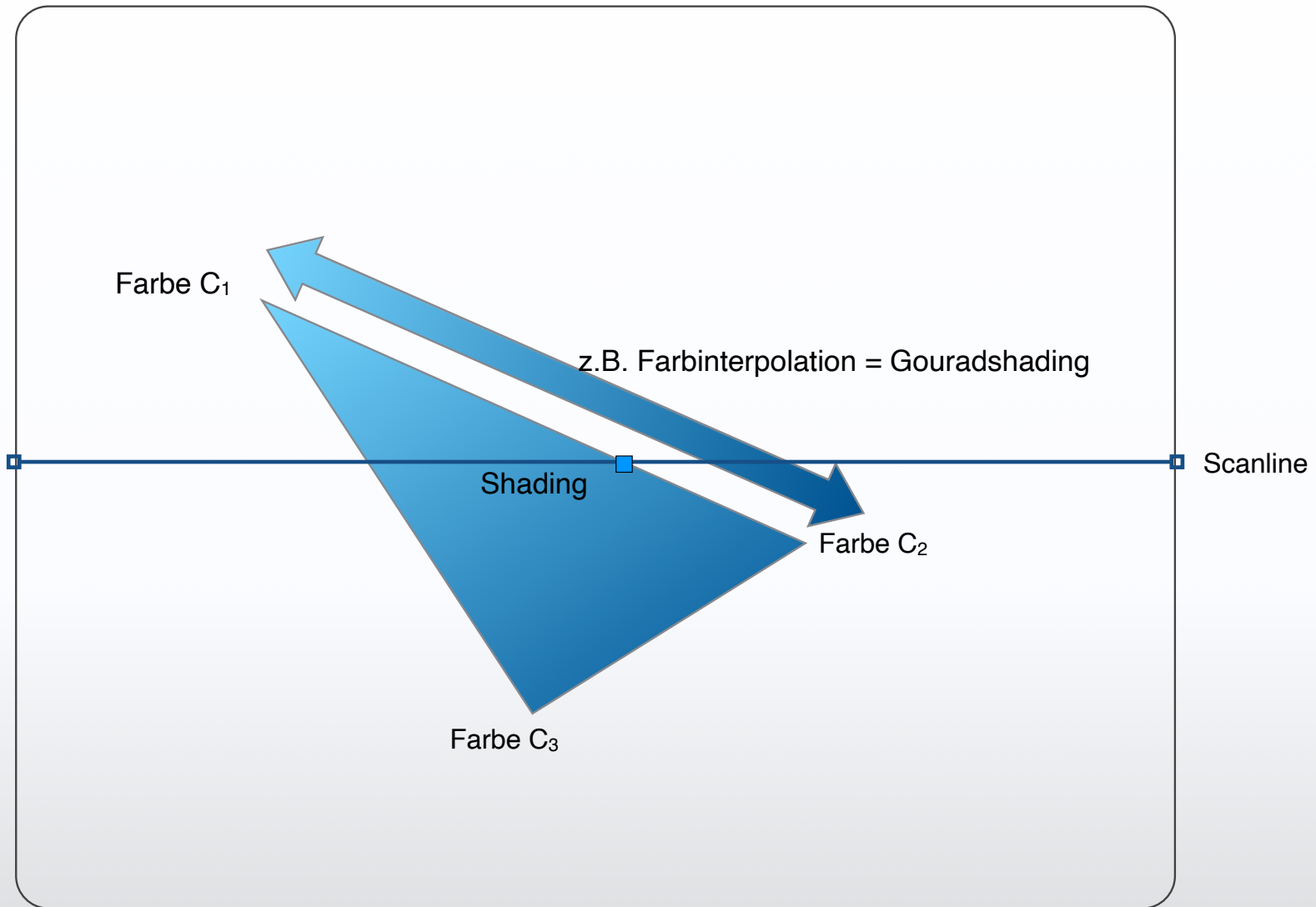
Shading in 2D • after projection



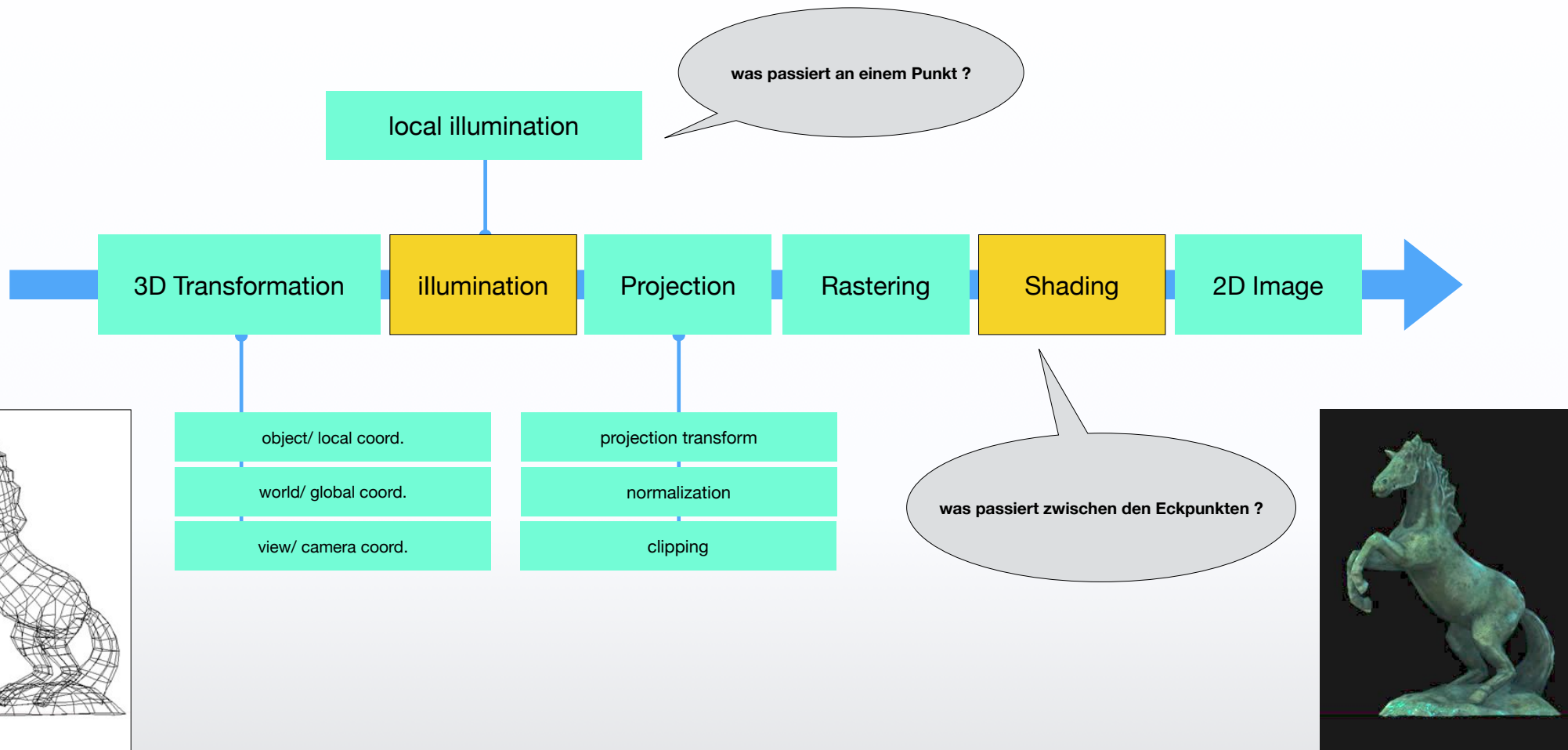
Shading 2D • after projection



Shading 2D • after projection



Fixed Graphics Pipeline



Phong, Blinn, Lambert

Lokales Beleuchtungsmodell

Berücksichtigt nur die Wechselwirkung
zwischen Lichtquellen / Objekt

Raytracing, Radiosity,...

Globales Beleuchtungsmodell

Berücksichtigt die Wechselwirkung
zwischen Lichtquellen / Objekt &
anderen Objekten

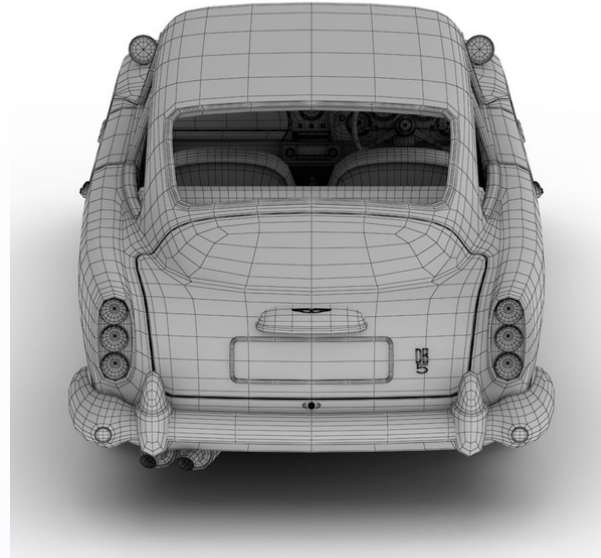
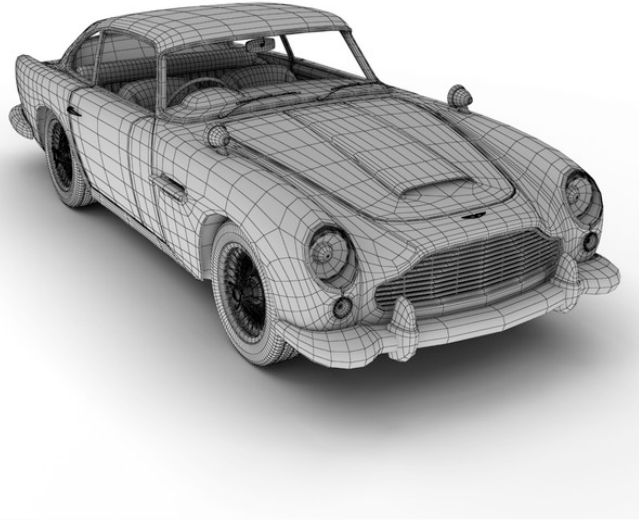
Illumination In einem Punkt



Shading Interpolation zwischen Vertices

Phong, Gouraud, Flat

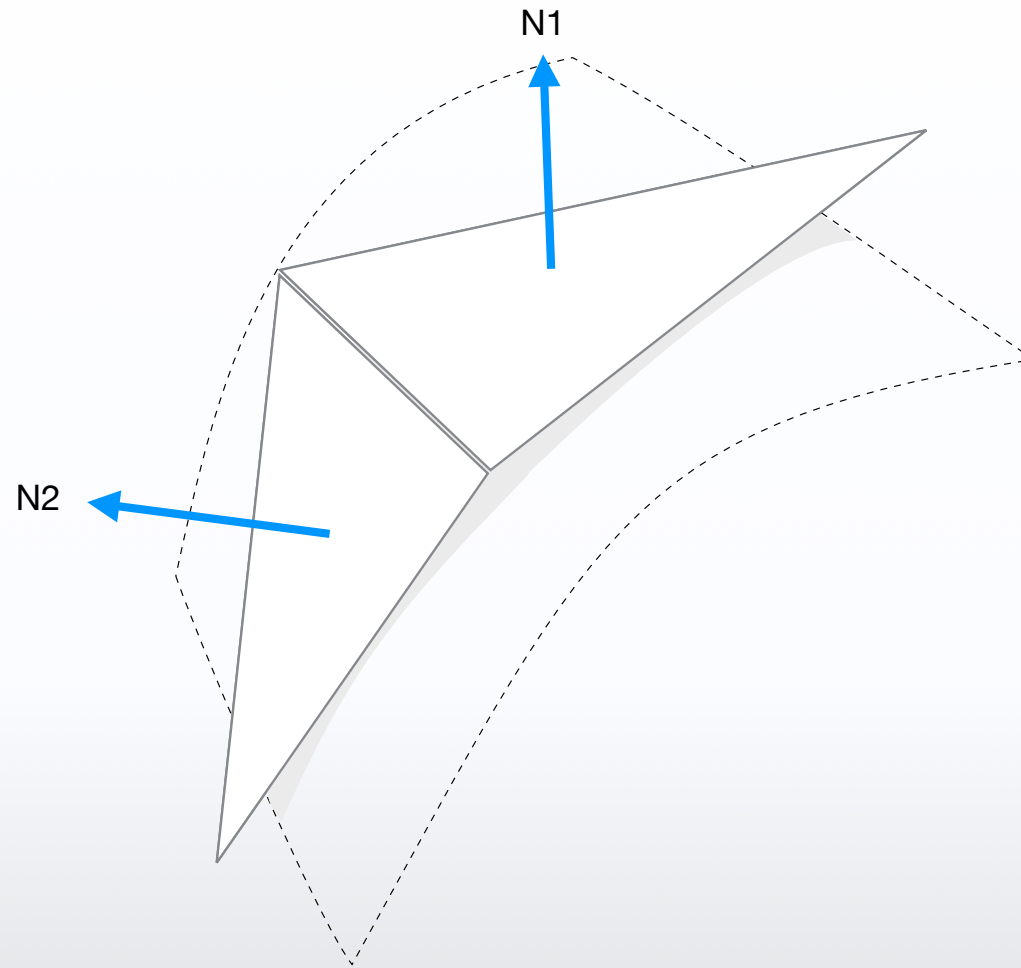
Curved Surface



Um komplexe Modelle in Echtzeit darstellen zu können
benötigen wir vereinfachte Darstellungen (mit weniger
Polygonen)
Zudem ist es das Ziel diese vereinfachten Modelle so gut
wie möglich darzustellen



Surface Normal



Berechnung der Normalen

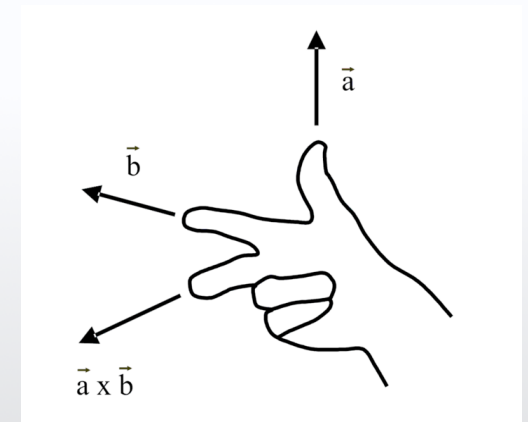
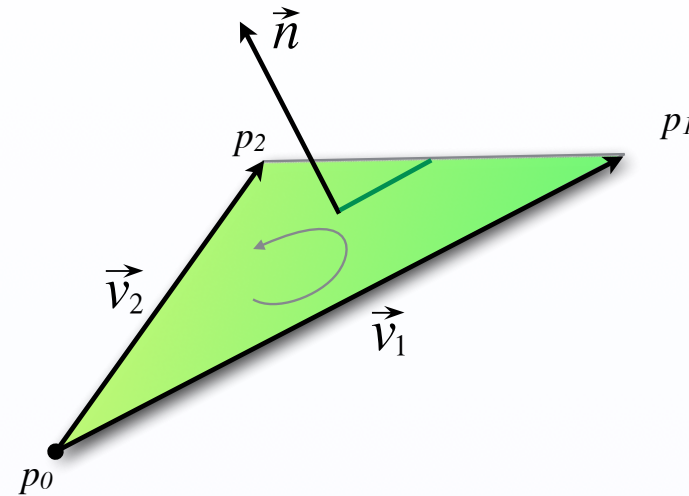
$$\vec{v}_1 = (p_1 - p_0)$$

$$\vec{v}_2 = (p_2 - p_0)$$

$$\vec{n} = \vec{v}_1 \times \vec{v}_2$$

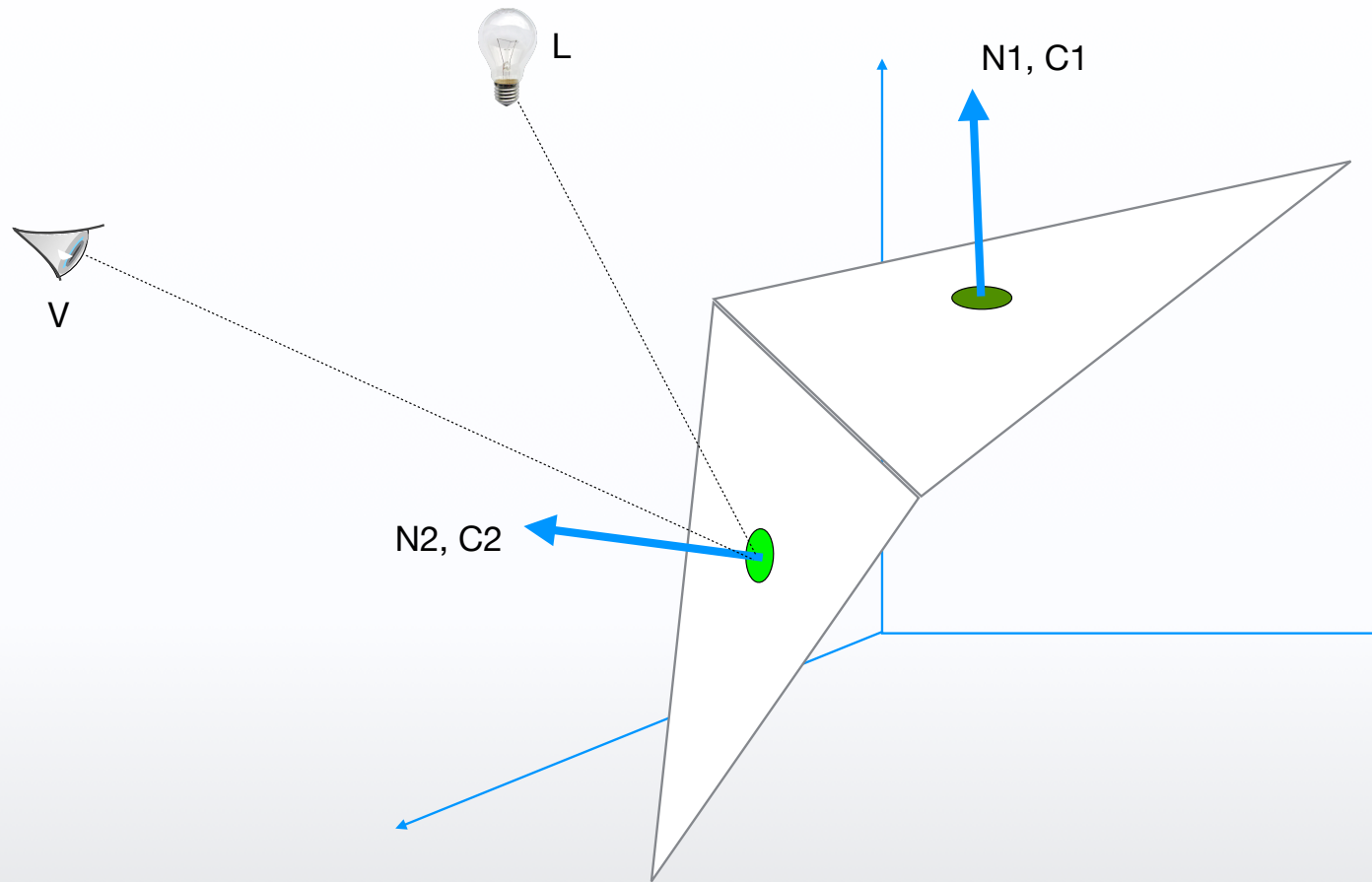
$$\bar{n} = \frac{\vec{n}}{|\vec{n}|} \quad \text{normalisierte Normale}$$

Drei Punkte spannen immer eine Ebene auf.
Es gibt daher nur eine Normale.



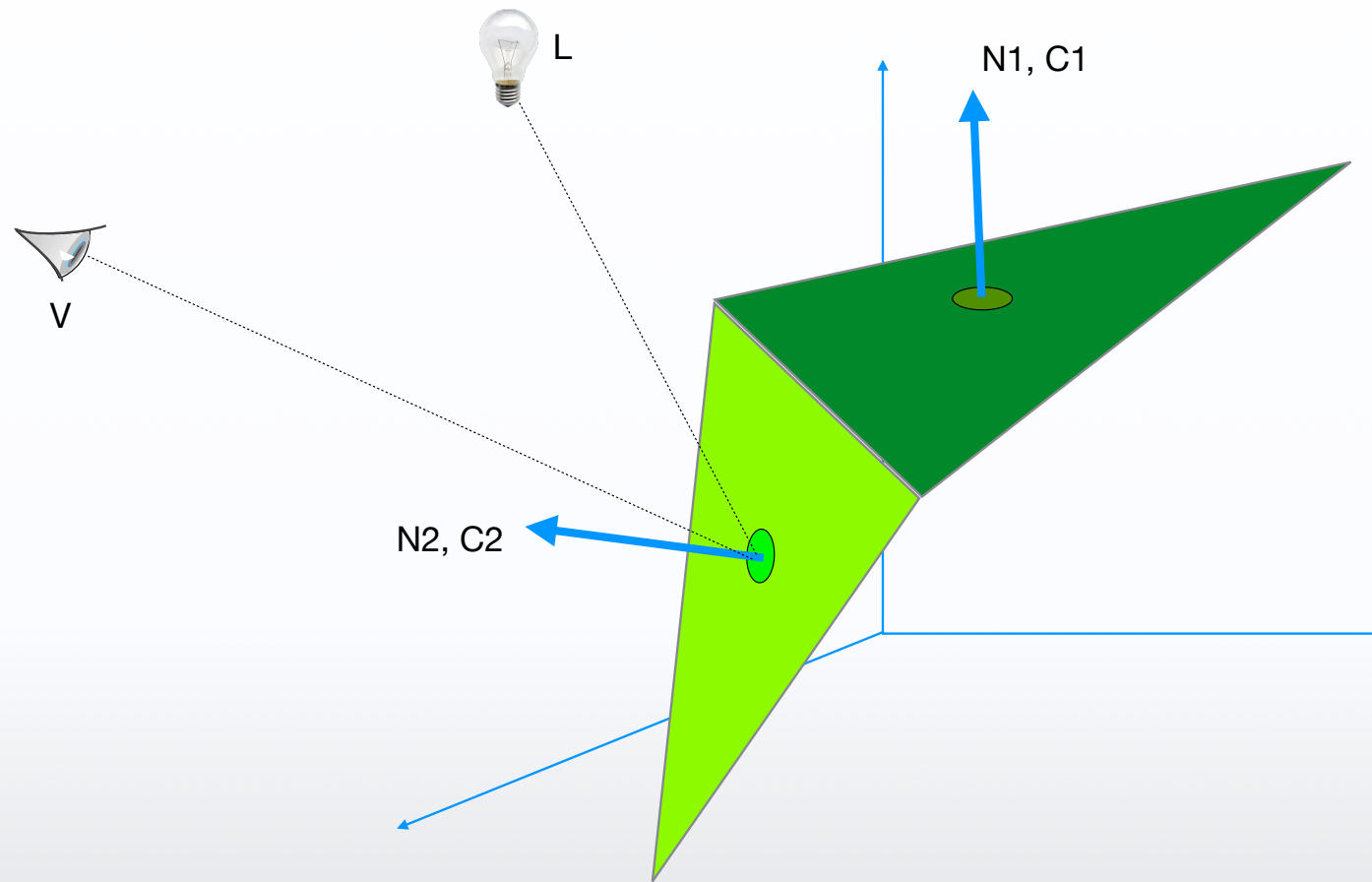
Kreuzprodukt ist nicht kommutativ !!

Surface Illumination



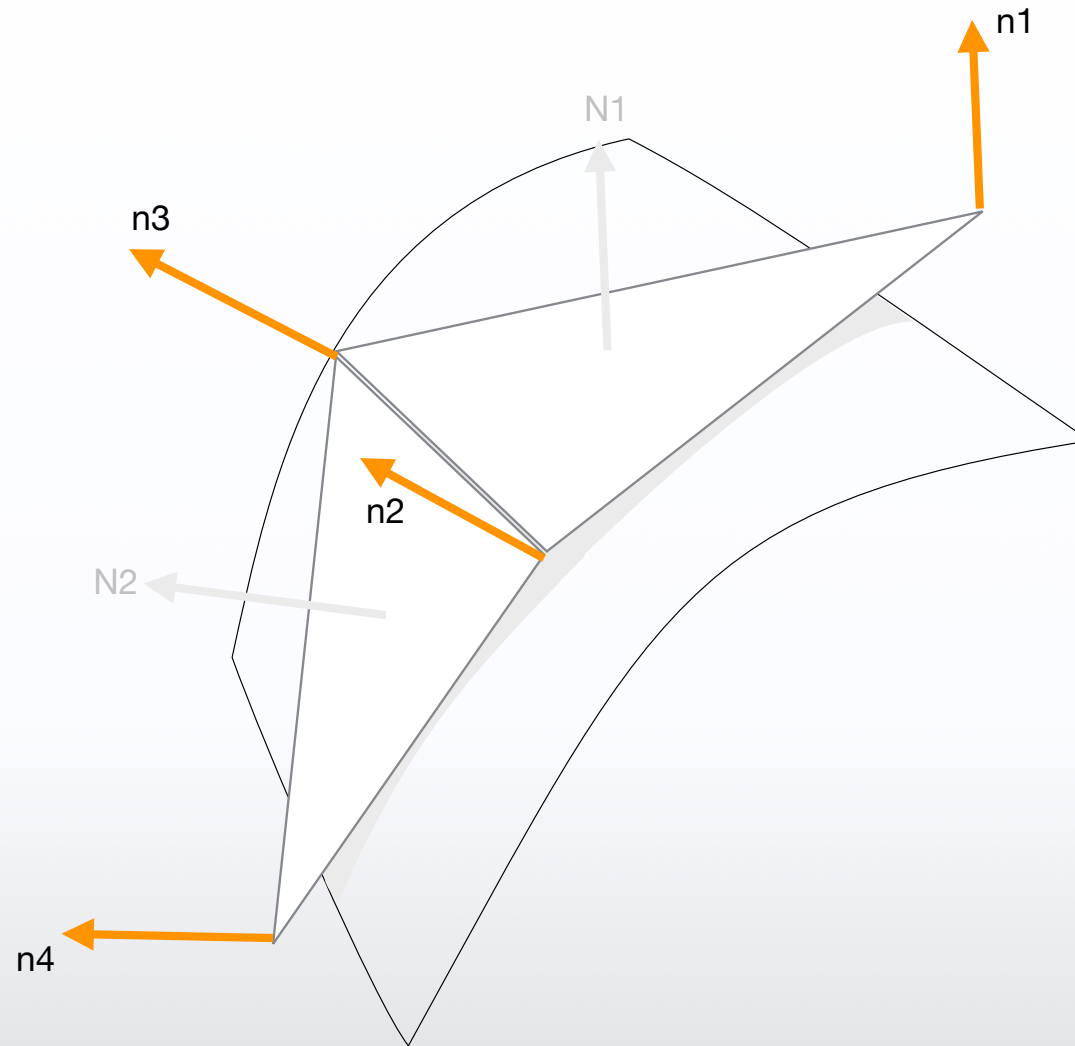
Light / Viewpoint / Normal : Color

Surface Illumination



Light / Viewpoint / Normal : Color

Vertex Normal

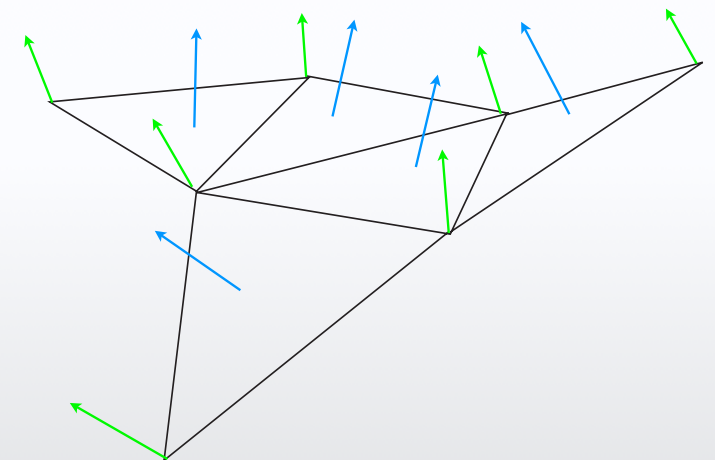
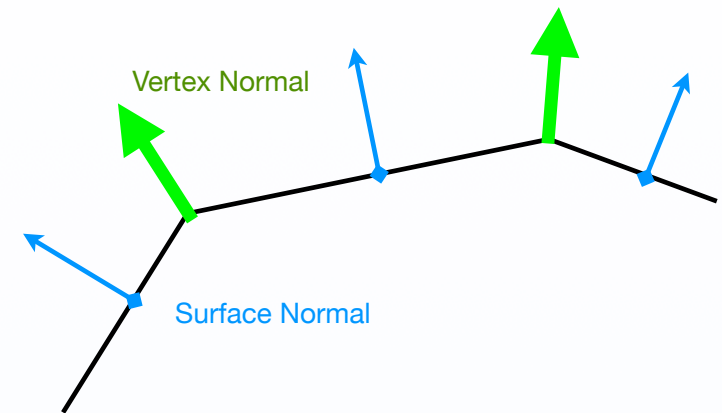


Berechnung der Vertex Normalen

- Eine Flächennormale ist zu wenig
- Einführung der Vertex Normalen
- Die Normale an einem Vertex wird aus dem Mittelwert der beteiligten Flächennormalen gebildet:

$$N_v = \frac{\sum_{Surface=1}^n N_{surface}}{\left| \sum_{Surface=1}^n N_{surface} \right|}$$

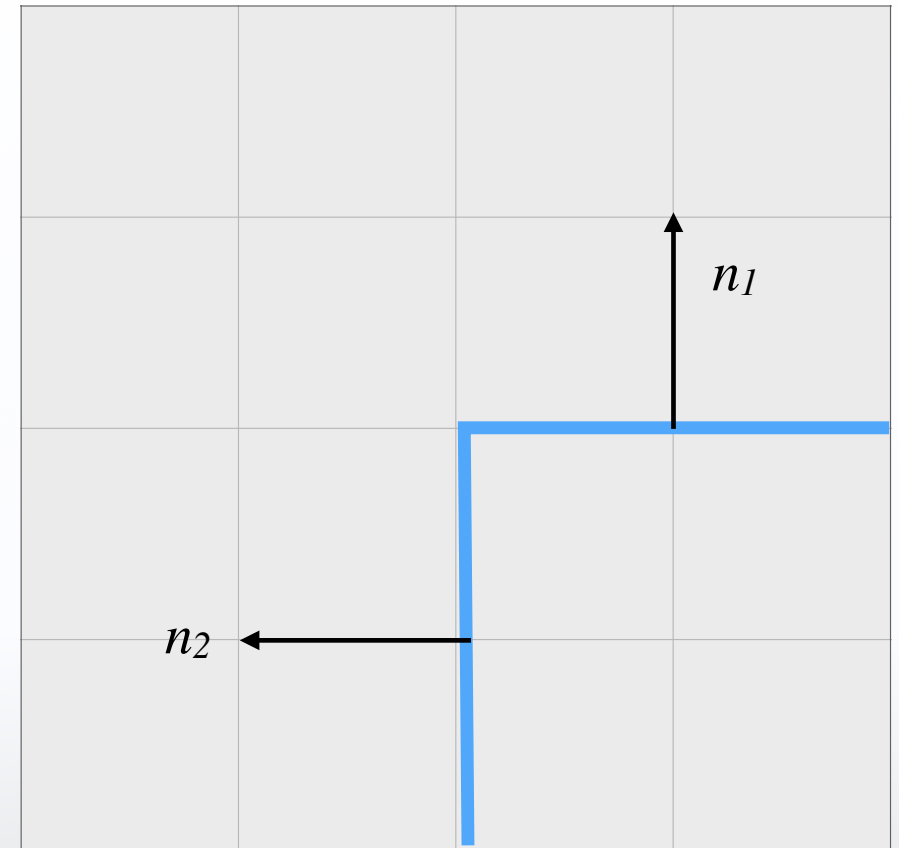
- Lichtberechnung an jeder Vertex Normalen
- Lambert/ Blinn/ Phong,...



Surface vs Vertexnormal

$$\vec{n}_1 = \begin{pmatrix} 0 \\ 1 \end{pmatrix}$$

$$\vec{n}_2 = \begin{pmatrix} -1 \\ 0 \end{pmatrix}$$

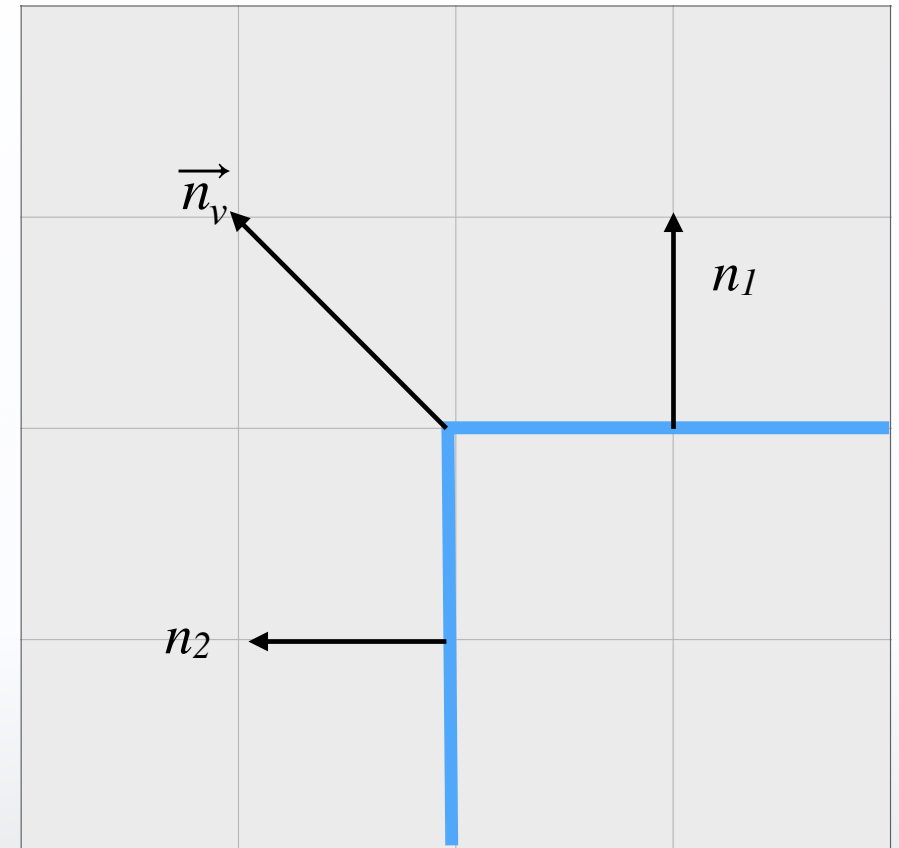


Surface vs Vertexnormal

$$\vec{n}_1 = \begin{pmatrix} 0 \\ 1 \end{pmatrix}$$

$$\vec{n}_2 = \begin{pmatrix} -1 \\ 0 \end{pmatrix}$$

$$\vec{n}_v = \begin{pmatrix} -1 \\ 1 \end{pmatrix}$$

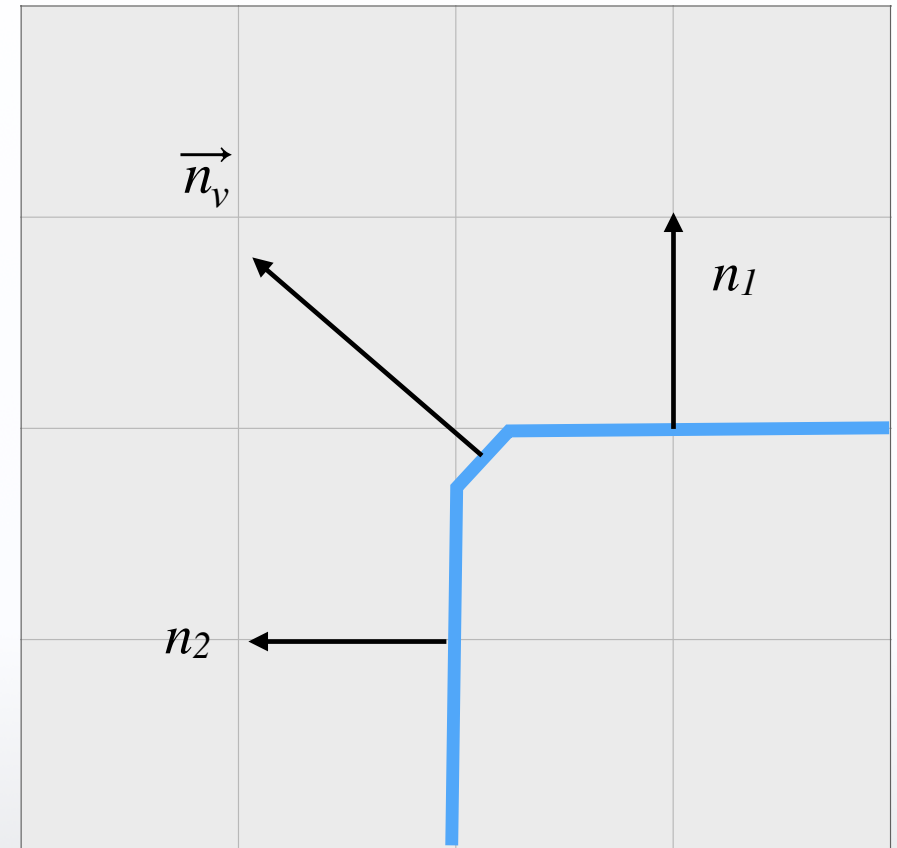


Surface vs Vertexnormal

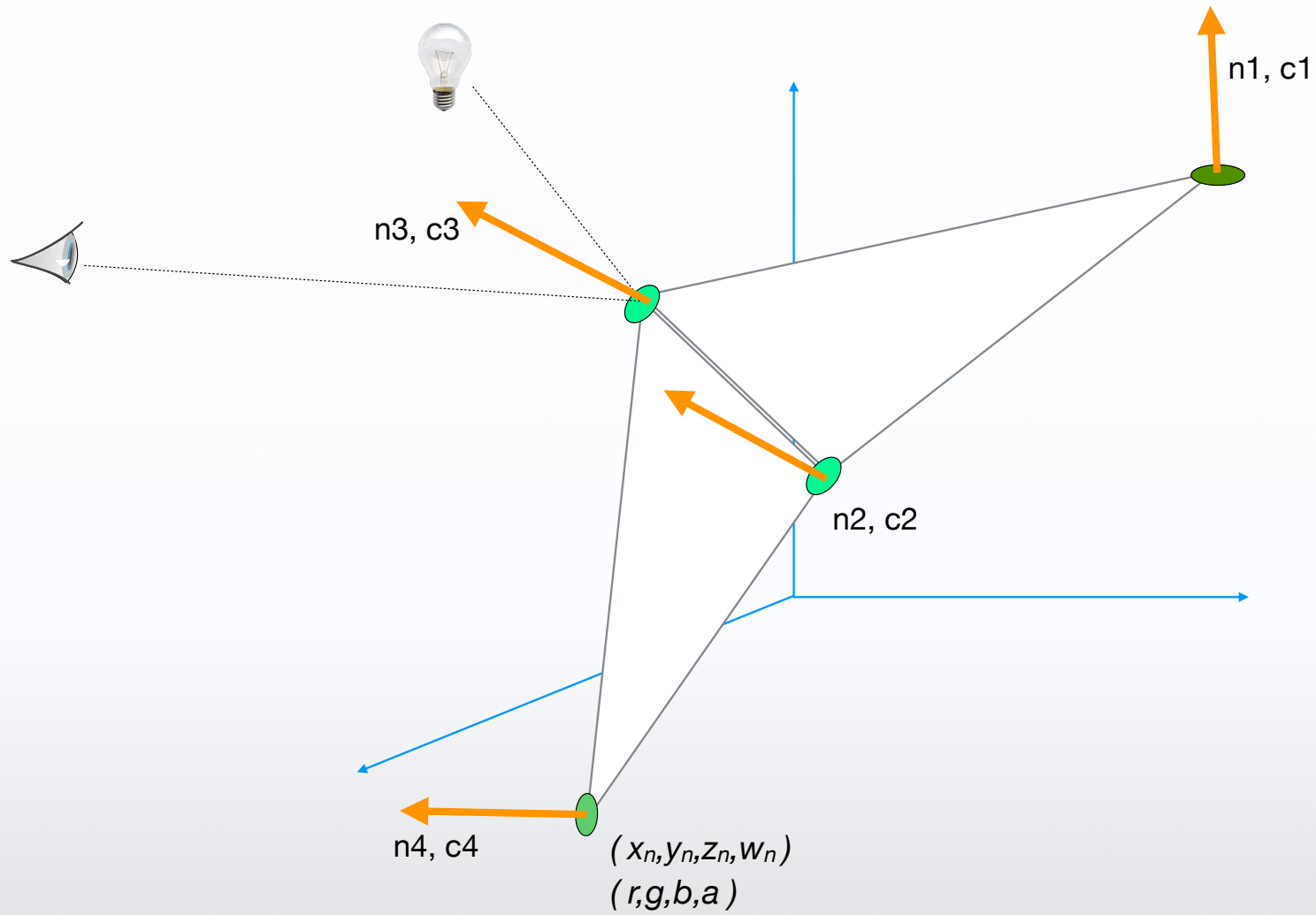
$$\vec{n}_1 = \begin{pmatrix} 0 \\ 1 \end{pmatrix}$$

$$\vec{n}_2 = \begin{pmatrix} -1 \\ 0 \end{pmatrix}$$

$$\vec{n}_v = \begin{pmatrix} -1 \\ 1 \end{pmatrix}$$



Vertex Illumination



Name.obj

This file uses centimeters as units for non-parametric coordinates.

mtllib teapot.mtl

v 0.067046 0.891365 -1.095709
v 0.000017 0.877831 -1.100676
v 0.000246 0.877585 -1.280246

....

vn 0.525486 -0.844128 0.106358
vn 0.500335 -0.865822 0.004225
vn 0.222867 -0.974796 0.010114
vn 0.234397 -0.968040 0.089202

.....

vt 1.567677 0.568903
vt 1.570830 0.577628
vt 1.544806 0.607388
vt 1.544792 0.581018
vt 1.531609 0.590698

....

usemtl phong1SG

f 2503/1/1 2430/2/2 2500/3/3 2499/4/4

f 2508/5/5 2446/6/6 2505/7/7 2504/8/8

f 2513/9/9 2436/10/10 2510/11/11 2509/12/12

.....

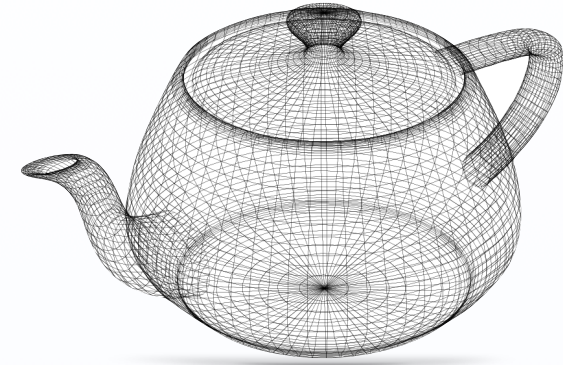
Material library

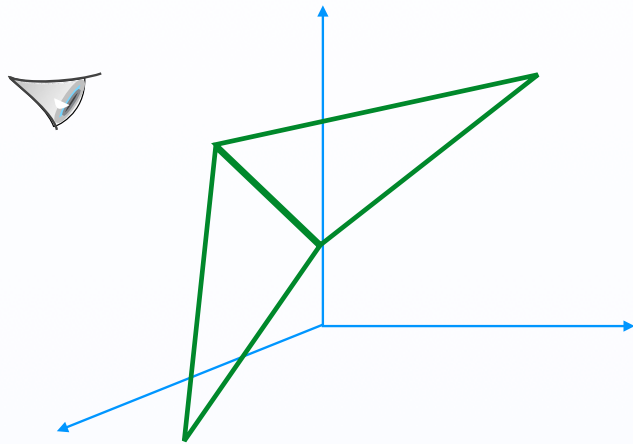
Vertice

Normals

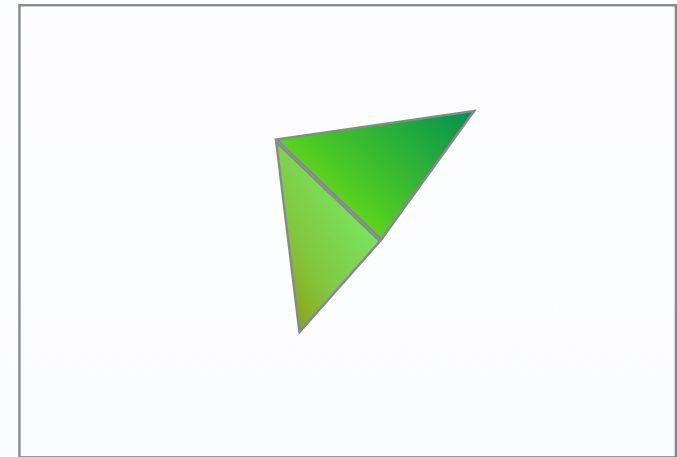
Texture

Face indices





PROJECTION



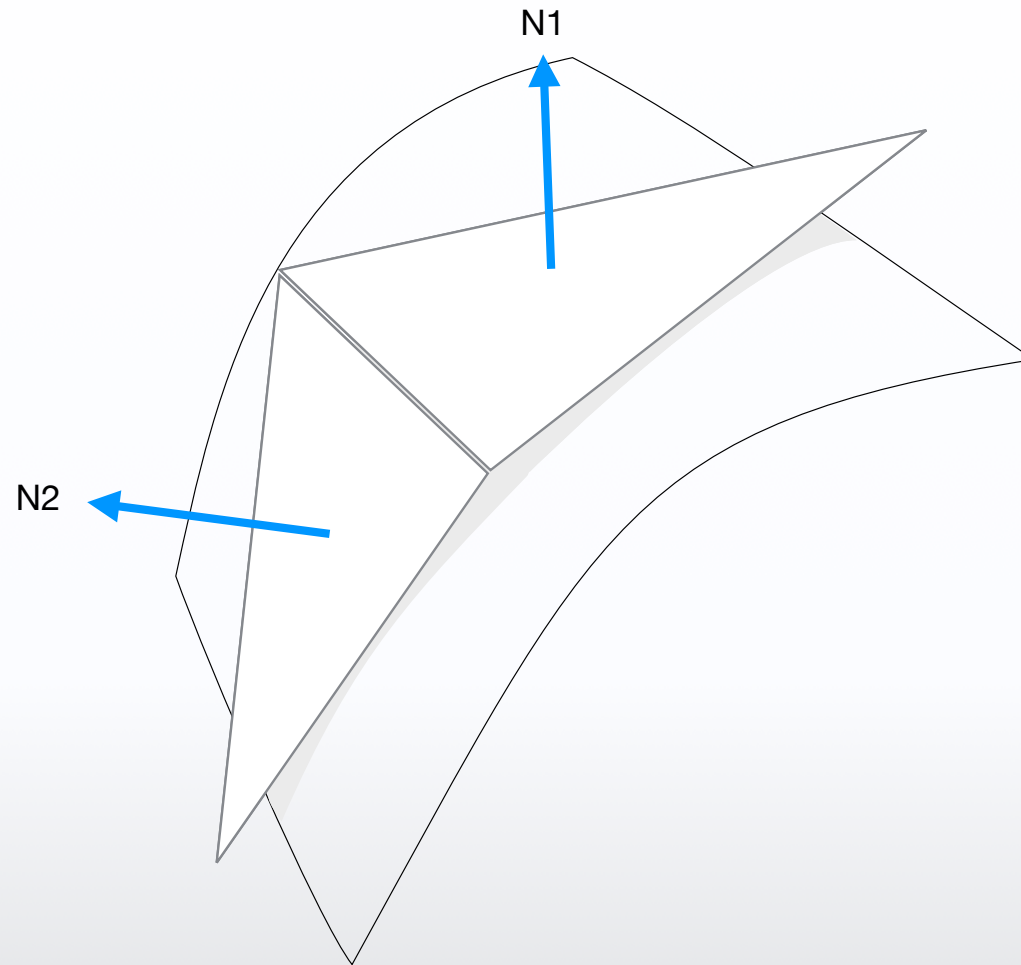
World

Vertex Farbwerte

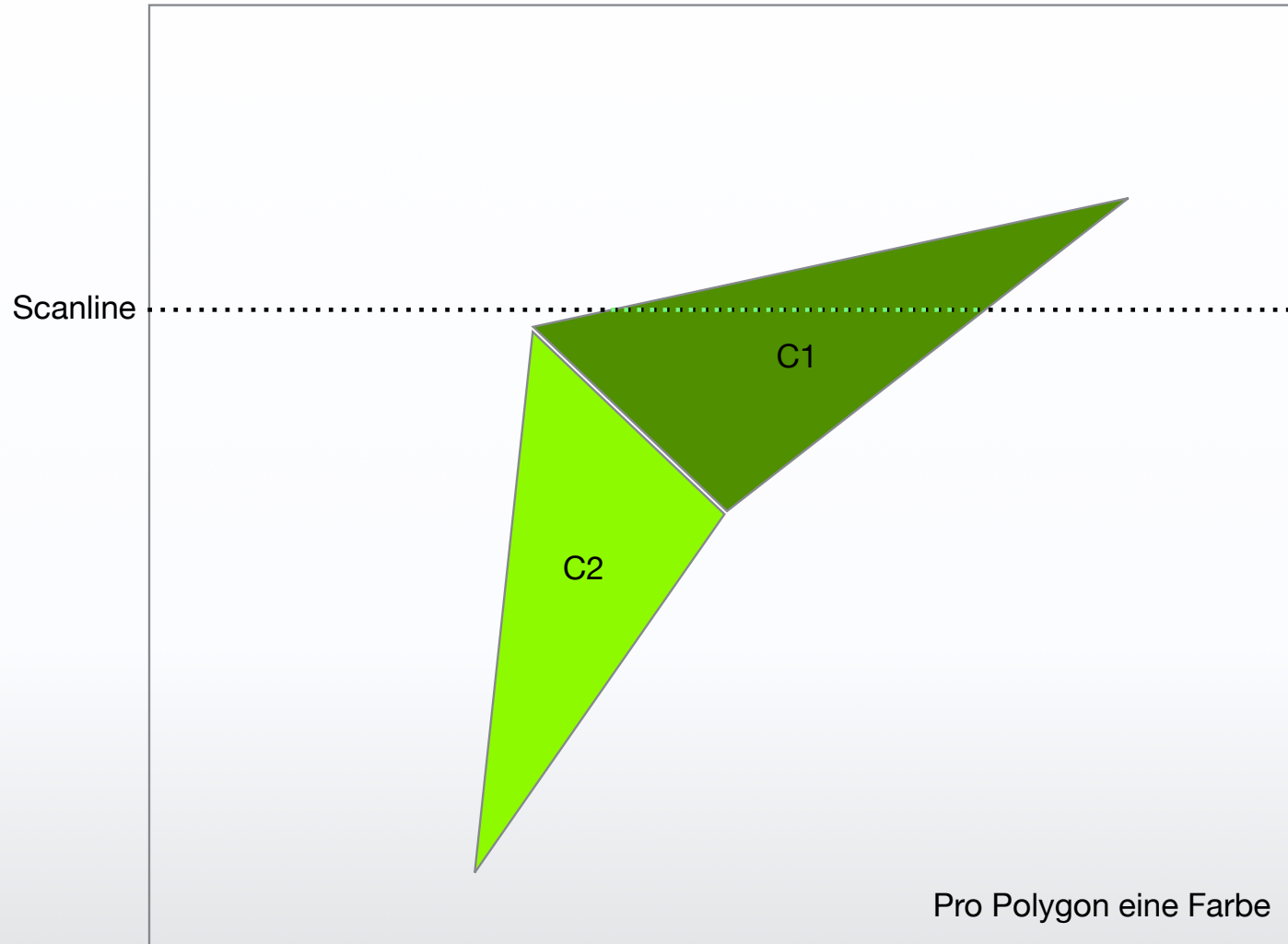
Window

Fläche füllen (Interpolation)

Surface Normal



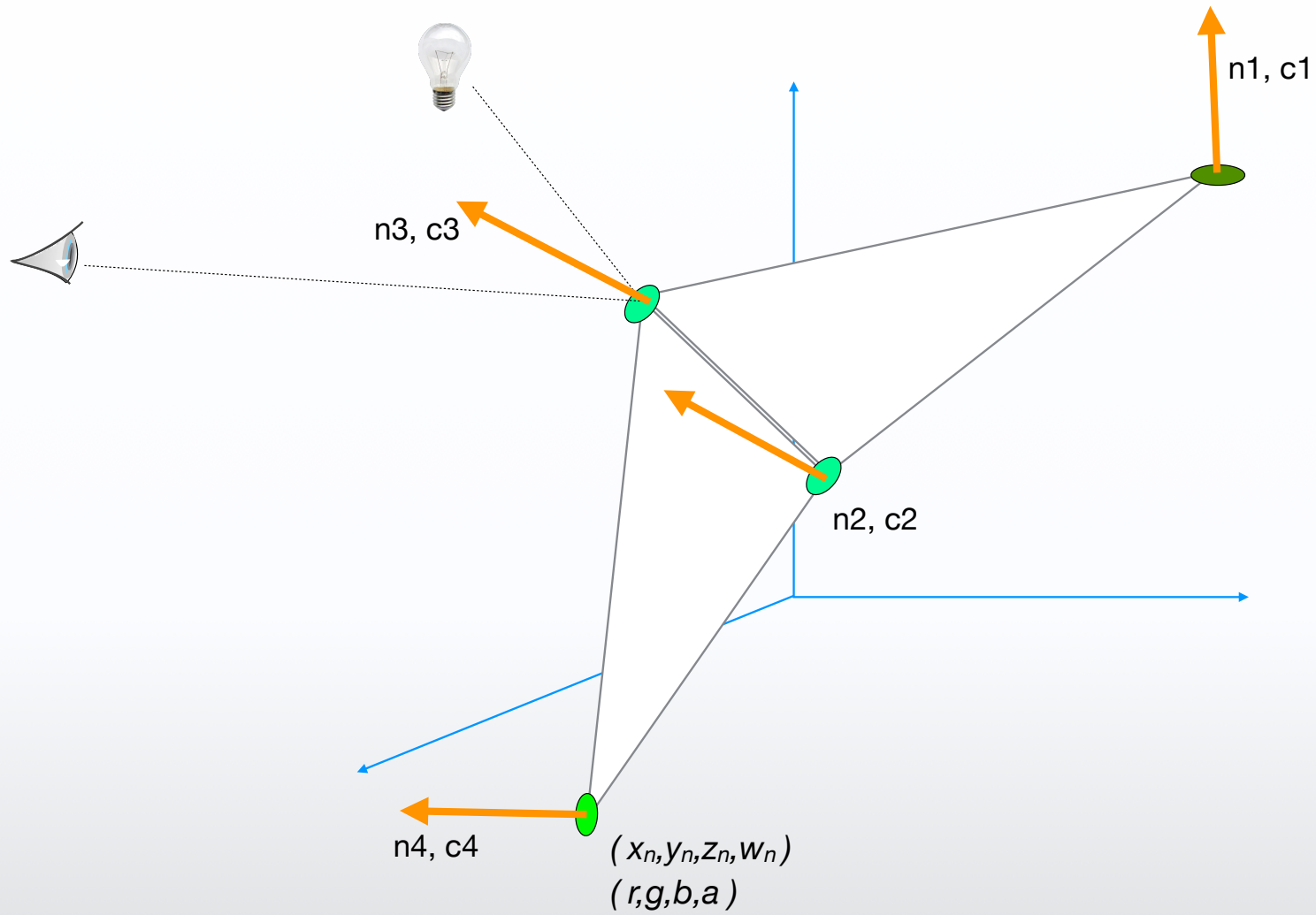
Flat shading



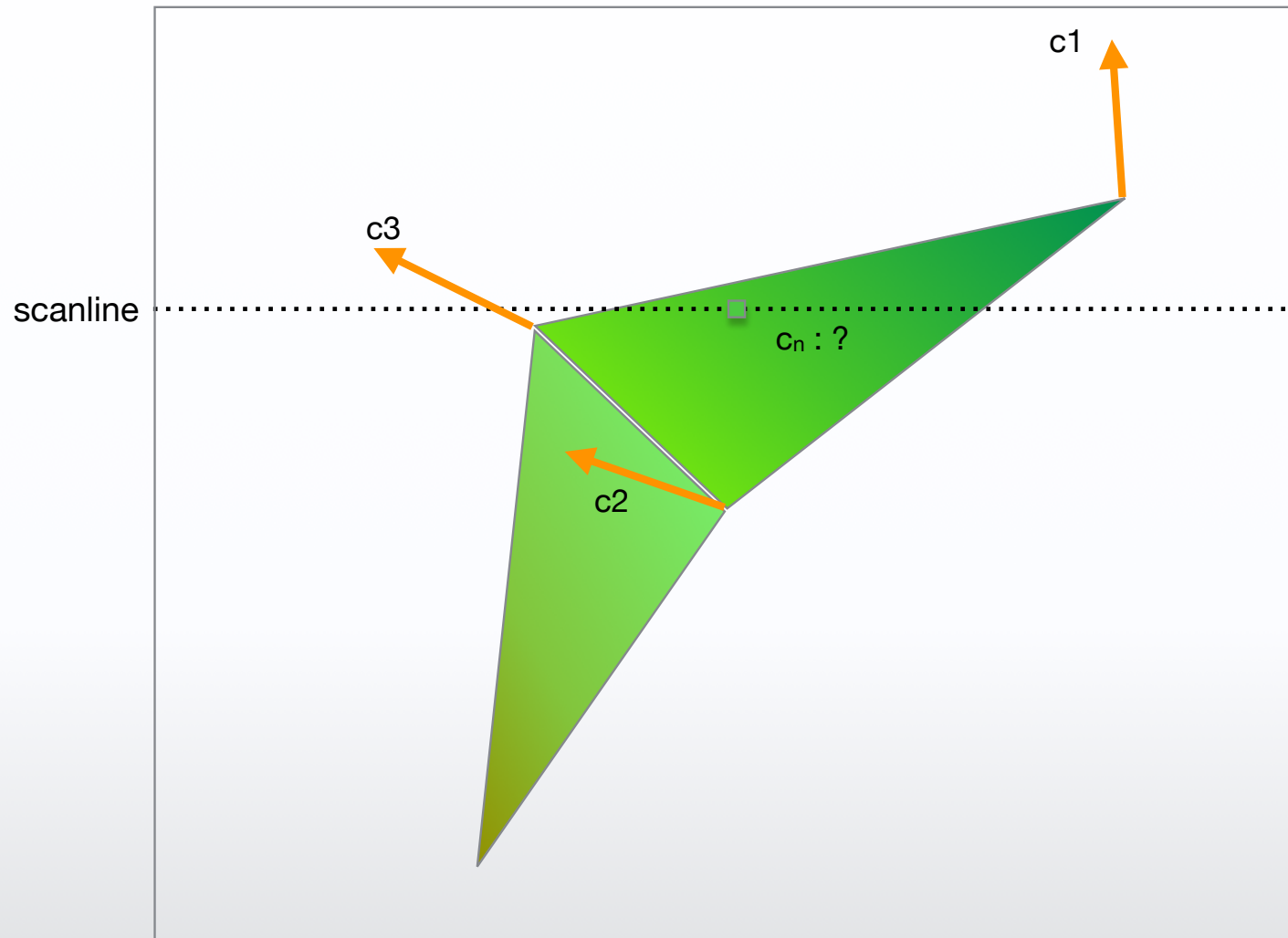
Flat Shading



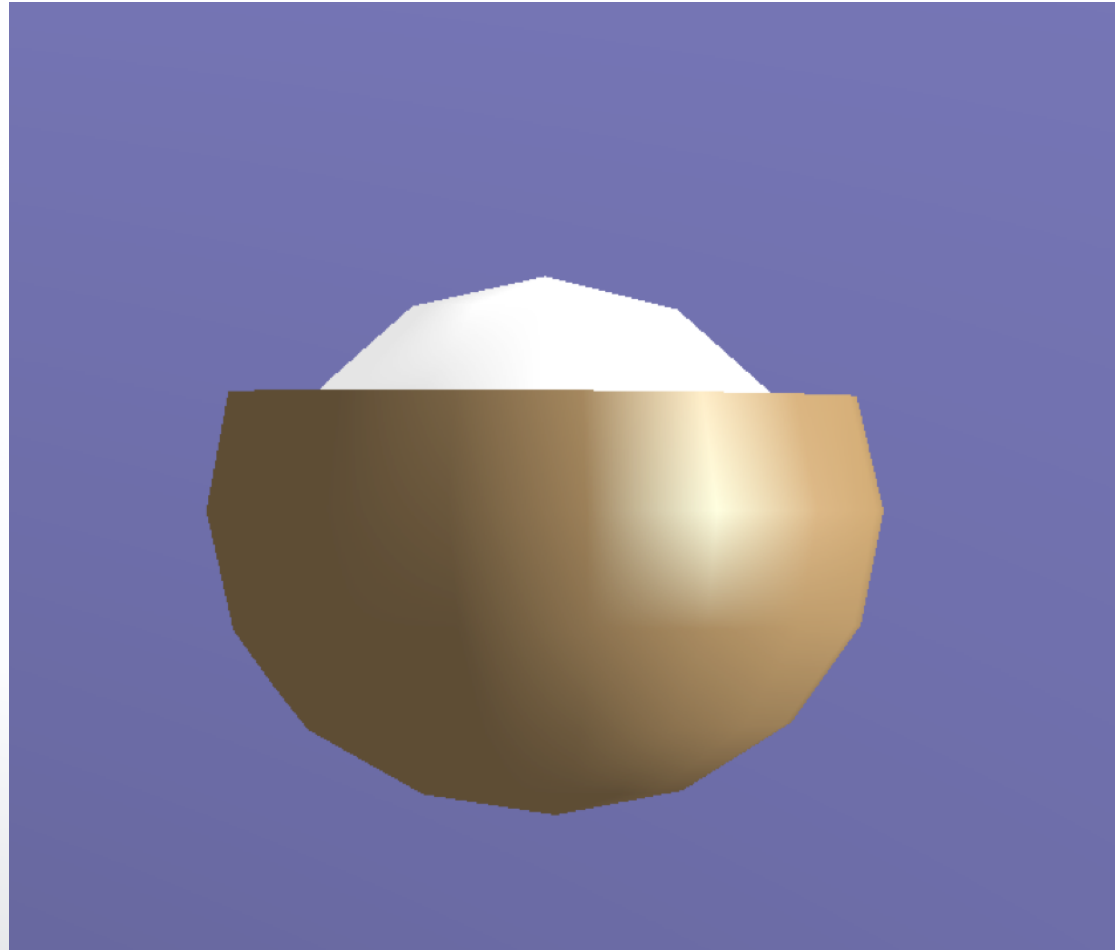
Vertex Illumination



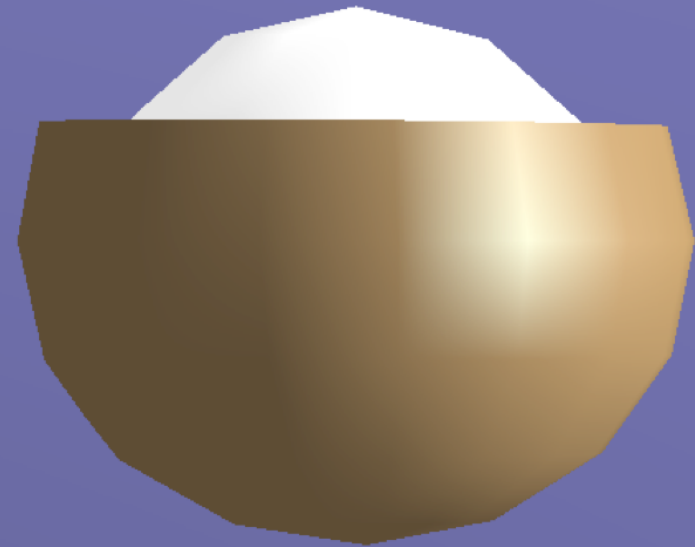
Gouraud Shading - color interpolation



Gouraud Shading

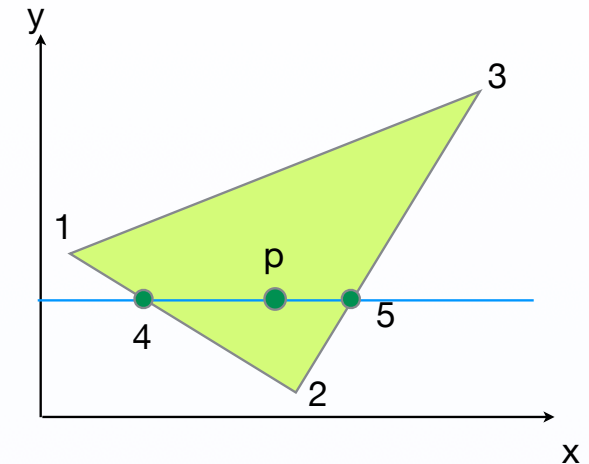


Flat vs Gouraud

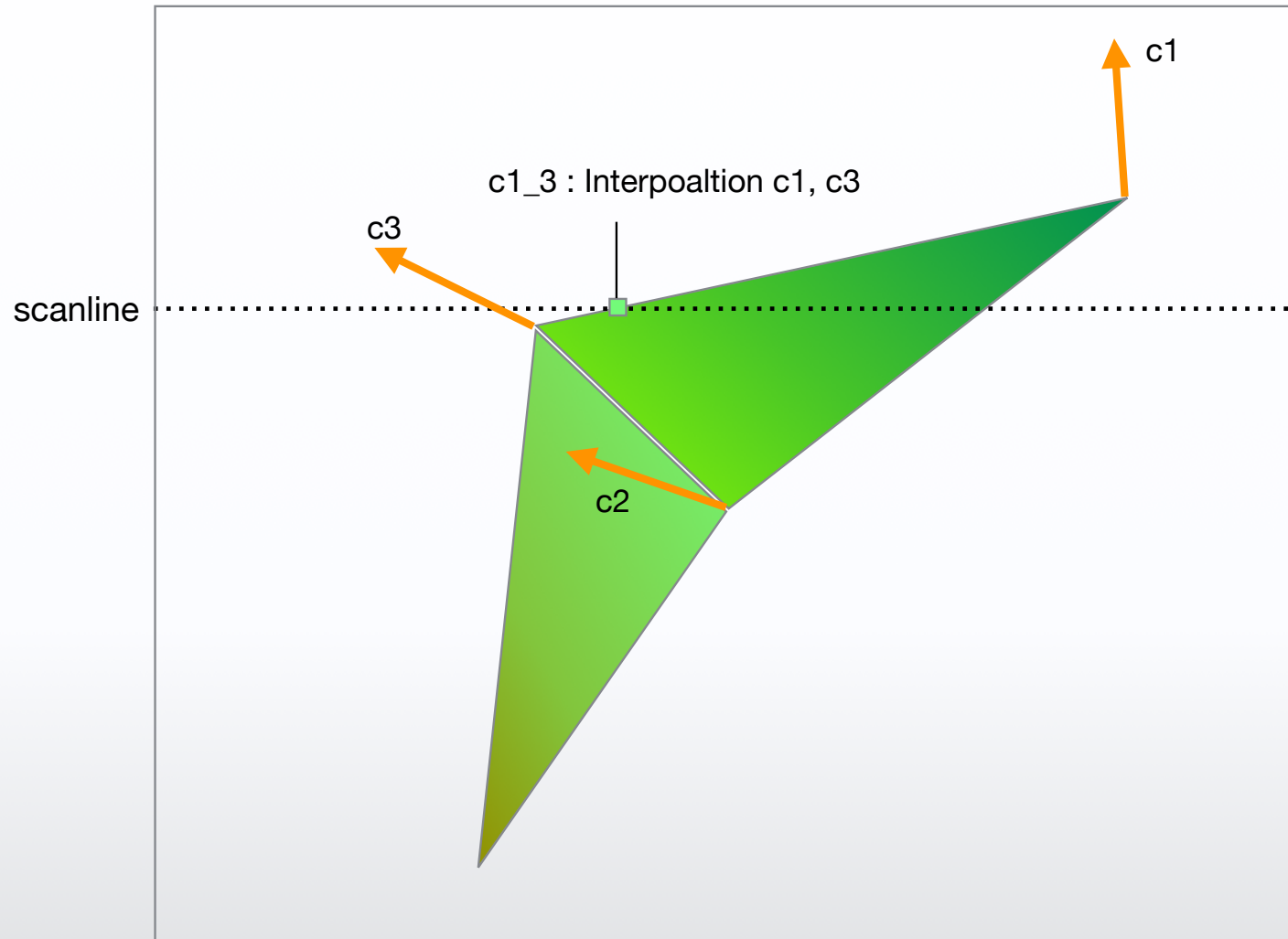


Gouraud Shading

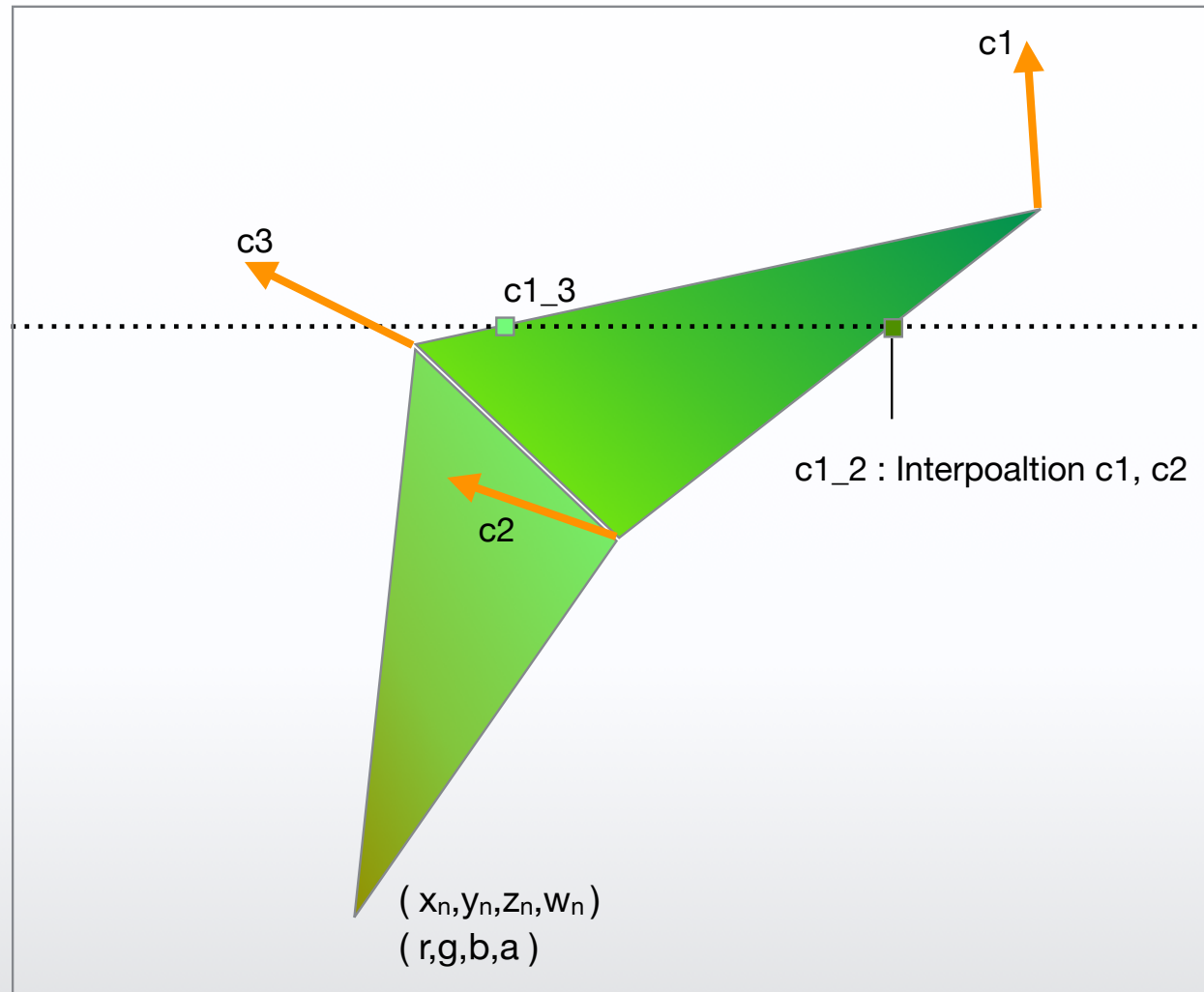
- Henri Gouraud, 1971
- Vertex Normale aus den Flächennormalen ermitteln
- Berechnung der Farbintensität an jedem Vertex (3 Werte anstatt einem Wert)
- Projektion der Szene (des Polygons) auf die Sichtebene
- Im Scanlinevorgang
 - Verdeckte Polygone verwerfen
 - Polygonkanten zeichnen
 - Füllalgorithmen
- Lineare Interpolation zwischen den jeweiligen Farbwerten



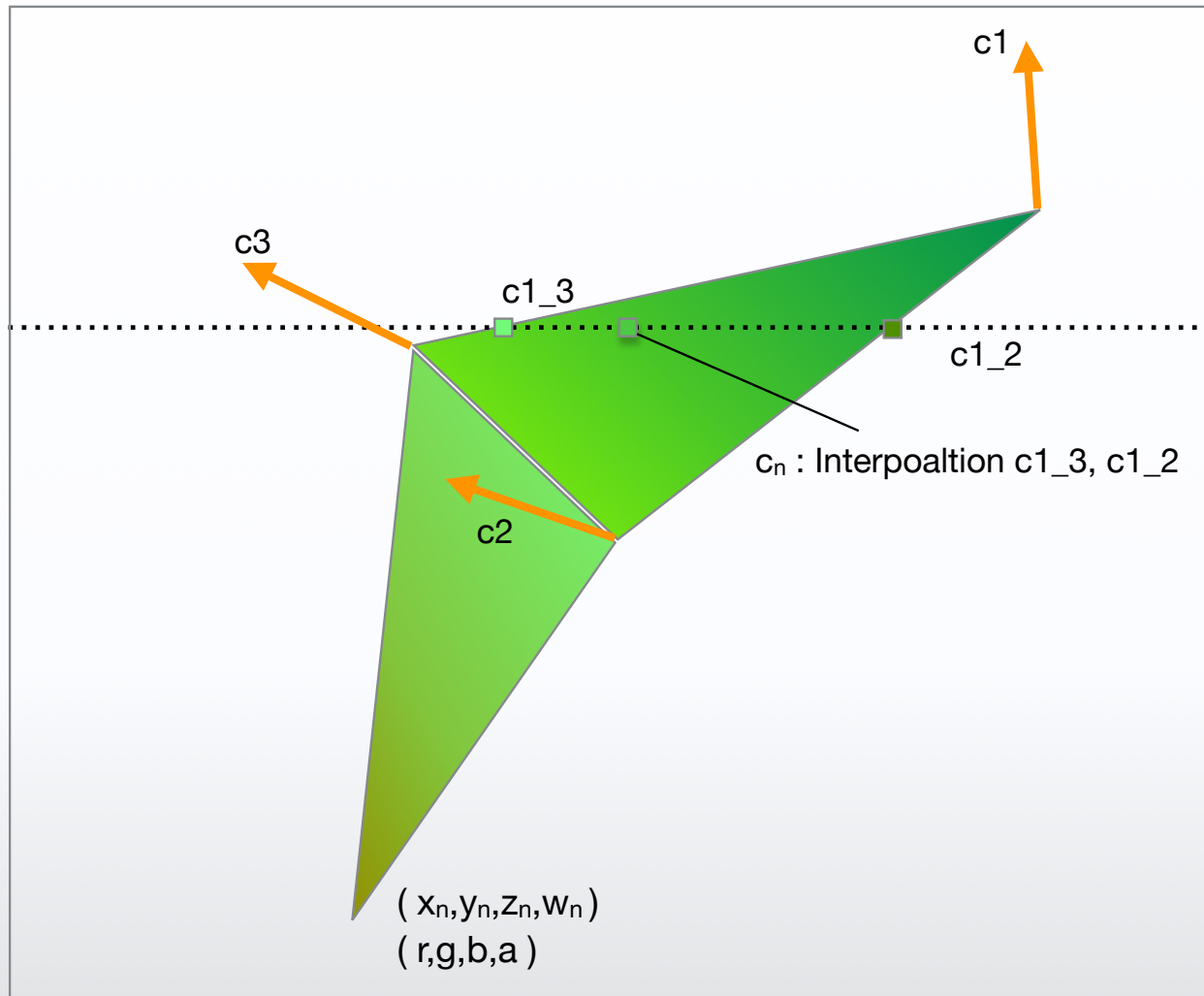
Gouraud Shading - color interpolation

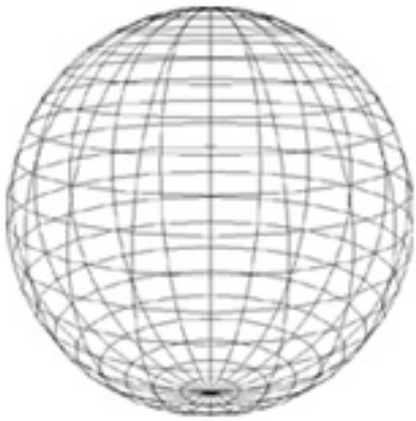


Gouraud Shading - colorinterpolation



Gouraud Shading - colorinterpolation





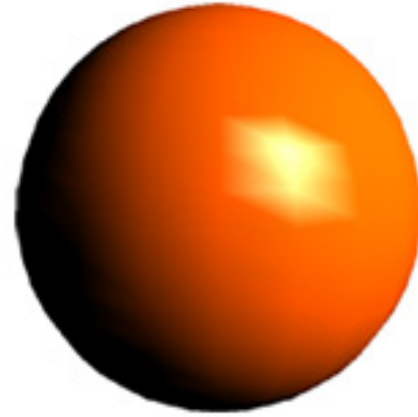
(a)

Wireframe



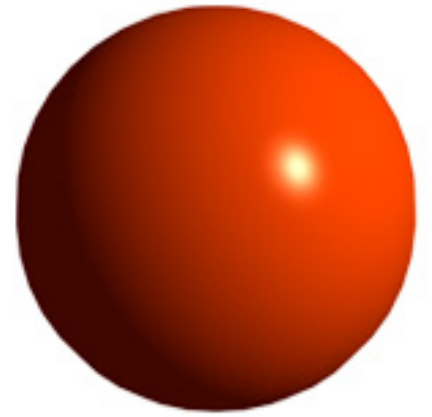
(b)

Flat shading



(c)

Gouraud shading

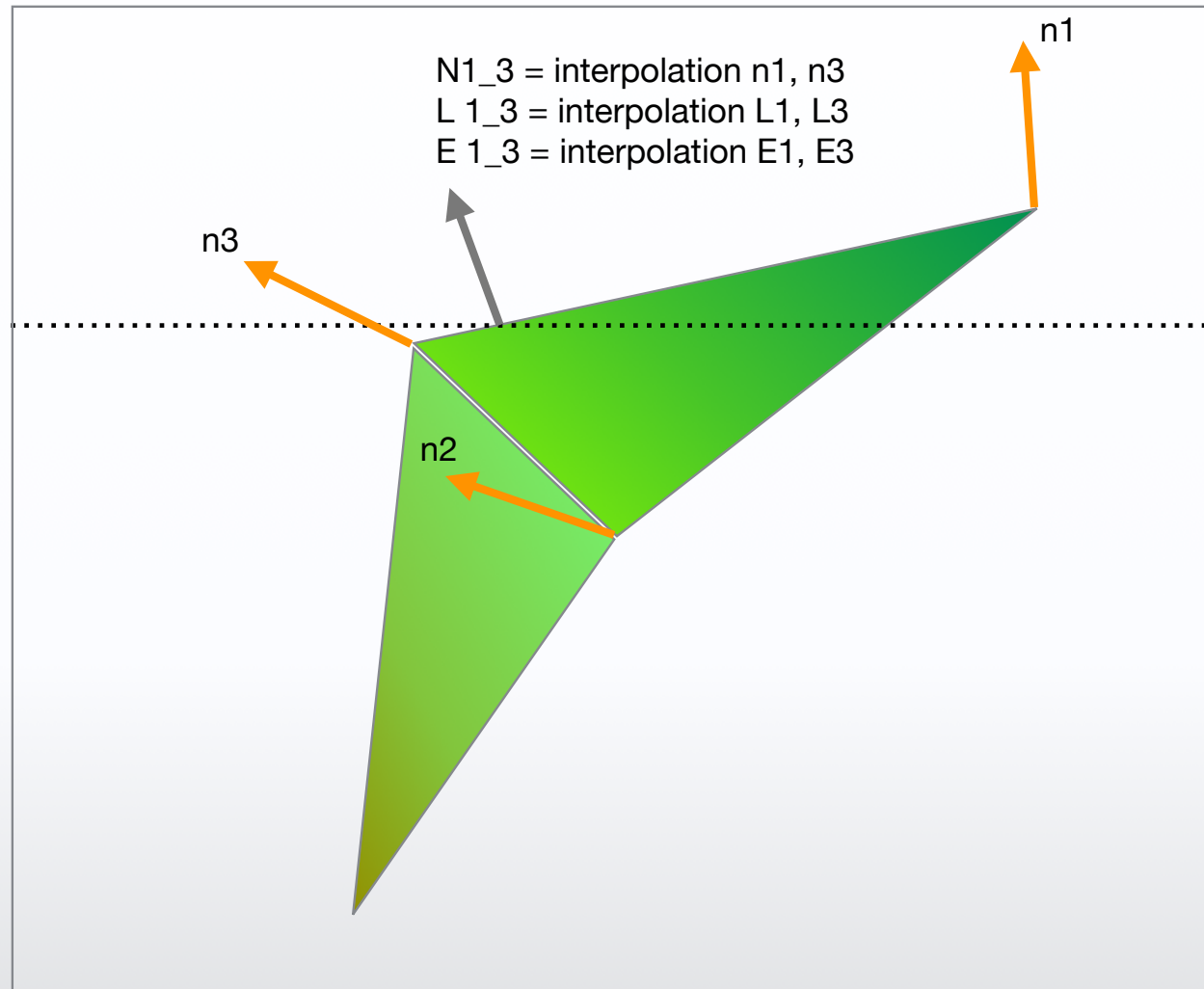


(d)

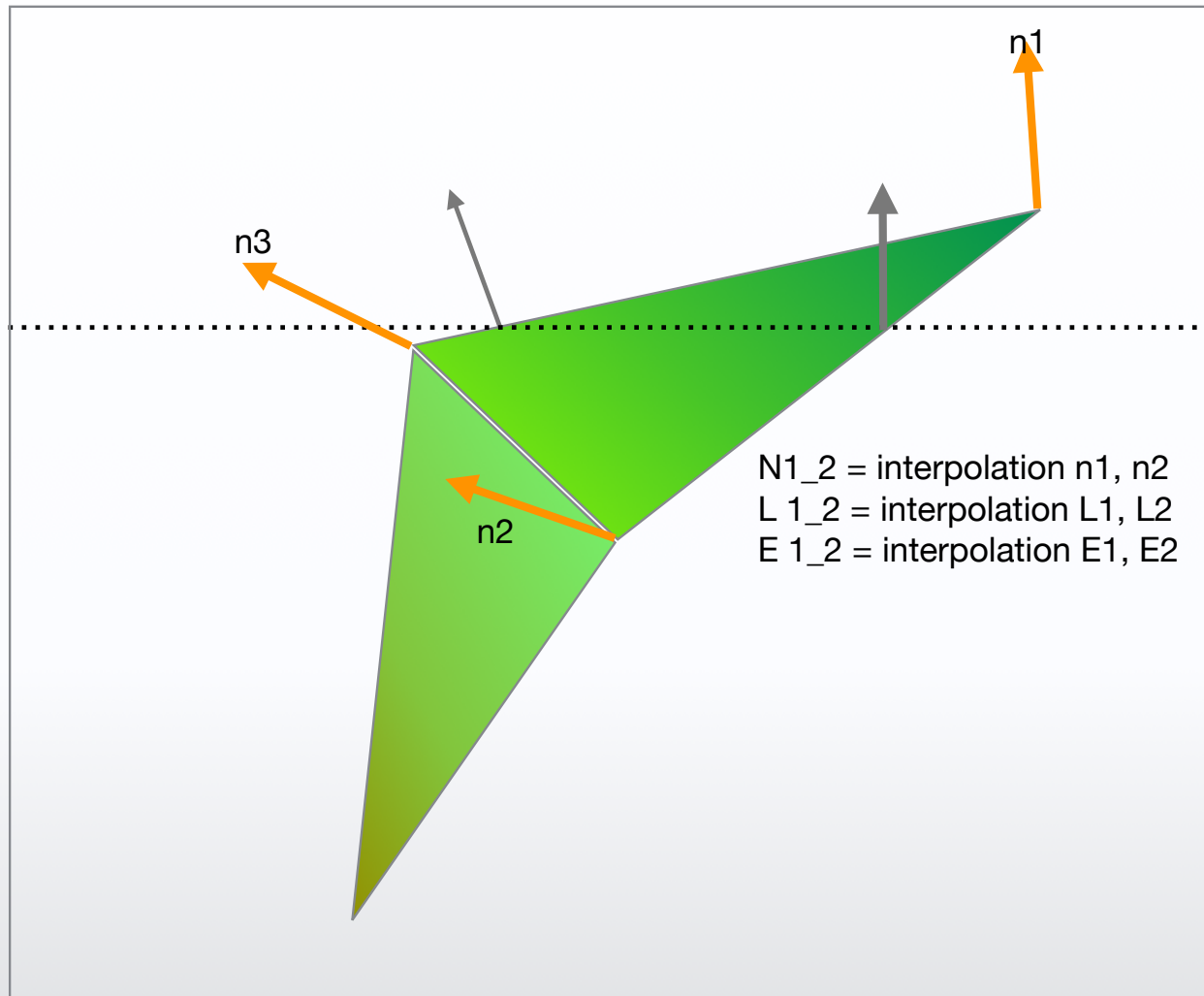
Phong shading

Phong Shading - normal interpolation

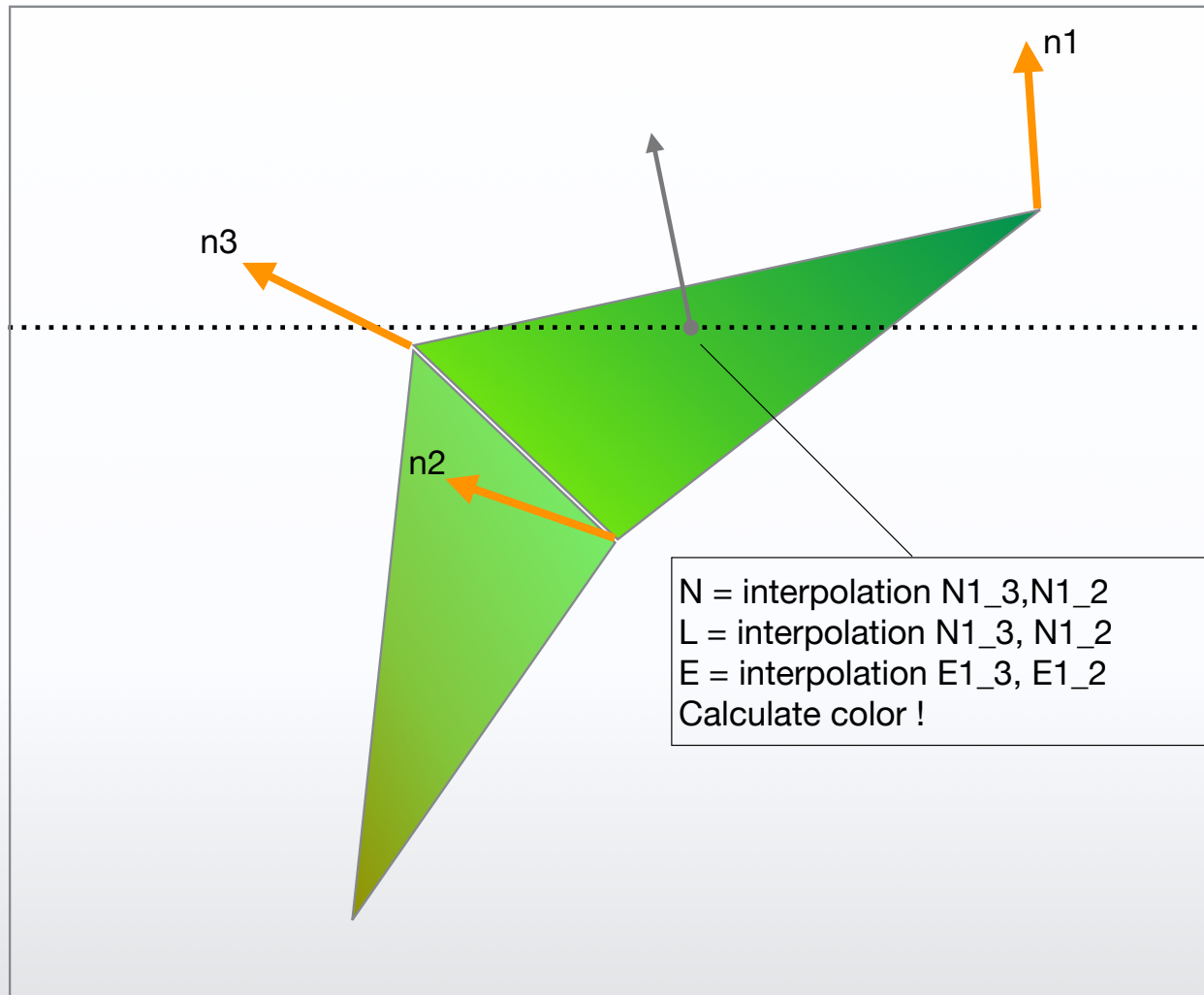
+
L = Lightvector
E = Eyevector



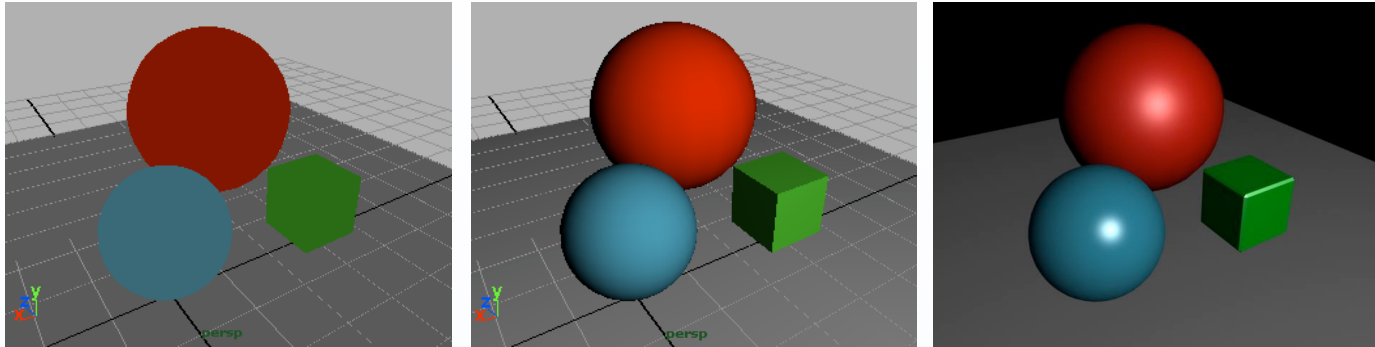
Phong Shading - normal interpolation



Phong Shading - normal interpolation

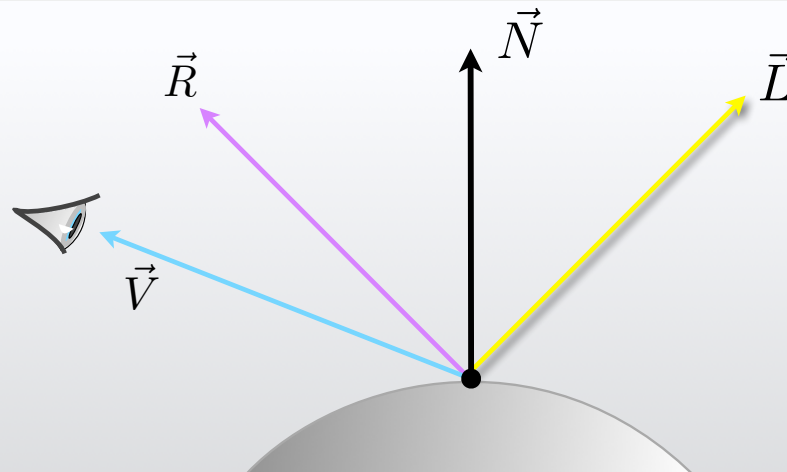


Phong Beleuchtungsmodell



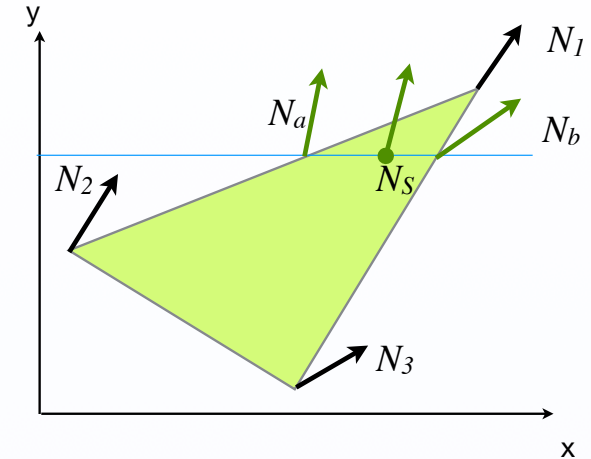
$$i_{ges} = i_a + i_d + i_s$$

$$i_{ges} = I_a \cdot k_a + \sum_{p=0}^m I_p \cdot [k_d(\overline{N} \cdot \overline{L}) + k_s(\overline{V} \cdot \overline{R})^n]$$



Phong Shading

- Phong Bui Tuong
- Vertexnormale aus den Flächennormalen ermitteln
- Projektion der Szene (des Polygons) auf die Sichtebene
- Interpolation der Vertex Normalen entlang der Kante
- Interpolation des View- und Lichtvektors entlang der Kante
- Interpolation der ermittelten Normalen auf der Scanline
- Illuminationsberechnung auf jede interpolierte Normale
- Punktweise Berechnung



Flat shading

- Der einfachste Fall (Flat shading)
- Berechnung der Normalen
- Berechnung der Farbintensität eines Pixels

■ der berechneten Normalen

■ Vektor zum Licht



Lambert

■ Reflexionsvektor des Lichtes

■ Vektor zum Betrachter



Phong

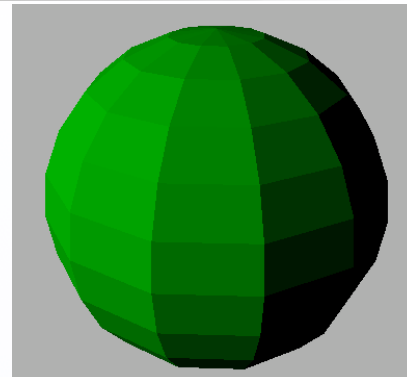
■ Halbvektor (Licht/Betrachter)



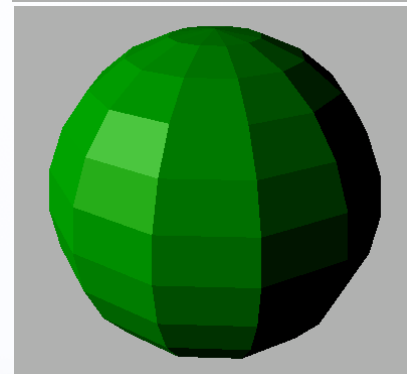
Blinn

- Die berechnete Intensität gilt dann für das gesamte Polygon

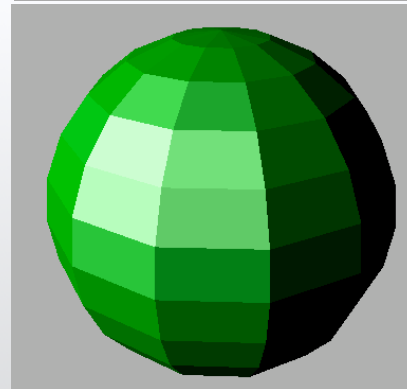
Flat_Lambert



Flat_Phong



Flat_Blinn



Interpolation

■ Problematik:

- Ist ein Pixel Innen oder Aussen?
- Ist ein Pixel auf der Kante (Edge) ? (Nonrealistic Rendering)
- Ist ein Pixel auf der Ecke (Vertex)? (Vertexshader)

■ Interpolation zwischen den Werten eines Polygons

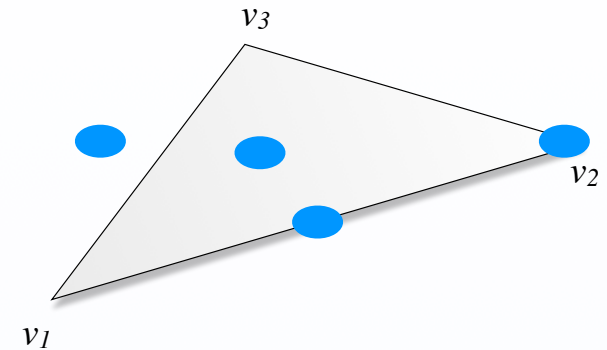
- Farbwerte
- Normalenwerte
- Texturwerte

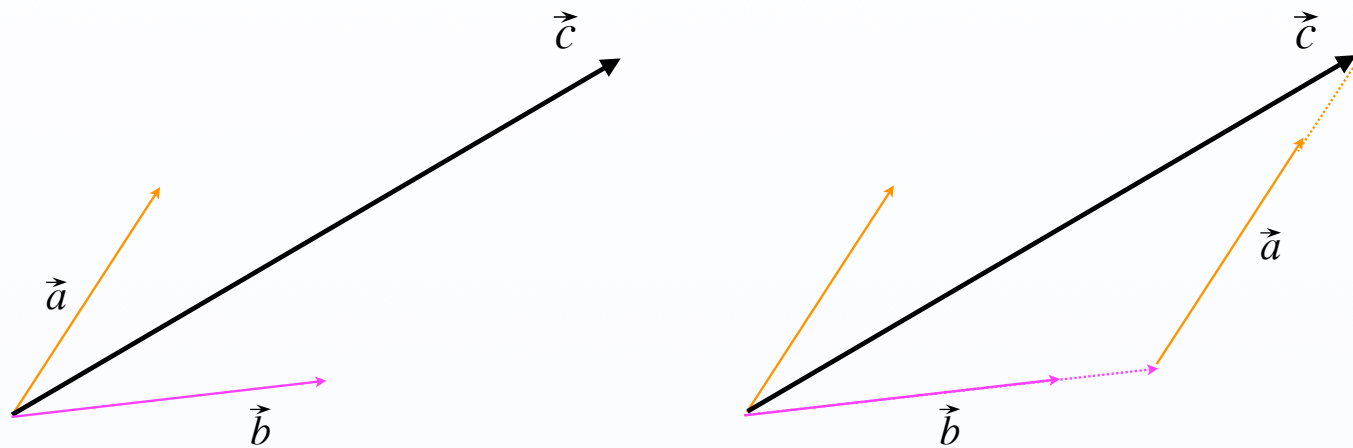
■ Wir brauchen eine Beschreibung eines beliebigen Punktes als Verhältnis von gegebenen Vertice (v_1, v_2, v_3)

■ Oft aufkommende Aufgabe in der Computergrafik

■ Die Lösung liegt im Universum:

■ BARYZENTRUM





Vektor $\vec{c}$ kann beschrieben werden als: $\vec{c} = \lambda\vec{a} + \mu\vec{b}$

α, β, γ sind keine Winkelangaben

Baryzentrische Koordinaten (2D)

$$p = a + \beta(b - a) + \gamma(c - a)$$

$$p = (1 - \beta - \gamma)a + \beta b + \gamma c$$

mit

$$\alpha \equiv 1 - \beta - \gamma$$

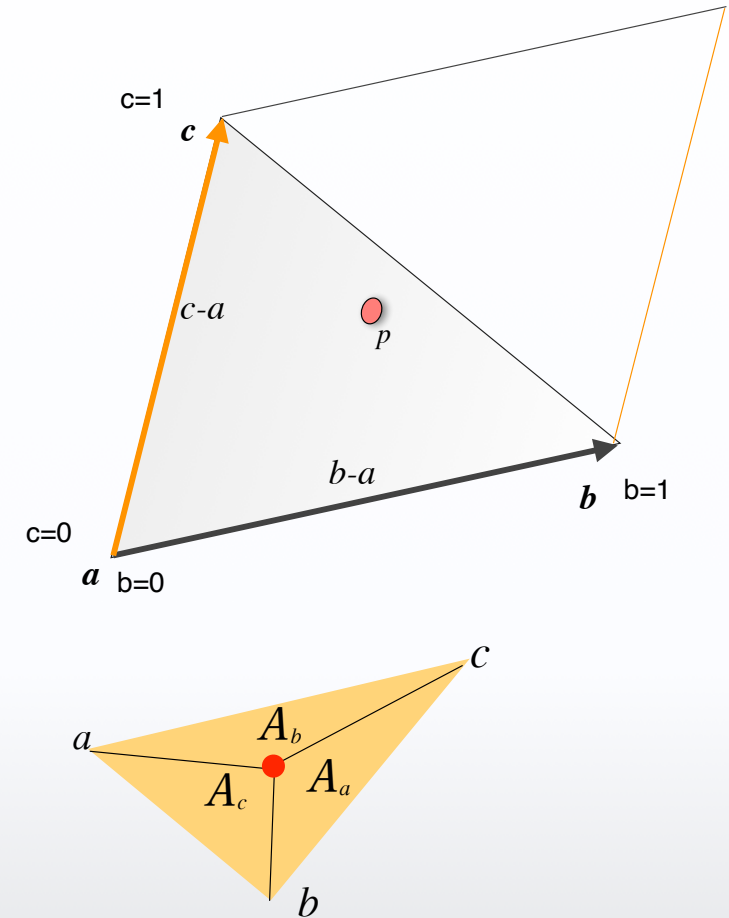
ergibt sich:

$$p(\alpha, \beta, \gamma) = \alpha a + \beta b + \gamma c$$

$$\alpha = A_a / A$$

$$\beta = A_b / A$$

$$\gamma = A_c / A$$

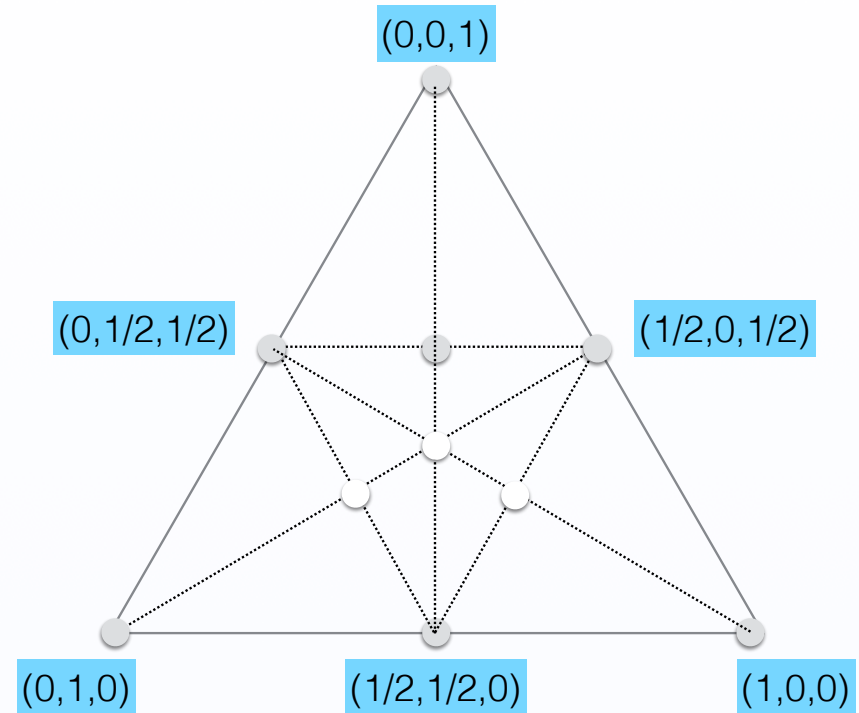


Baryzentrische Koordinaten

- Jeder Punkt p lässt sich bestimmen als:

$$p(\alpha, \beta, \gamma) = \alpha a + \beta b + \gamma c$$

mit $\alpha \equiv 1 - \beta - \gamma$



Innerhalb
wenn alle Koordinaten
zwischen (0,...,1) (Inside/
Outside-Test)

Kante (Edge)
Eine Koordinate 0
Die anderen zwischen (0,...,1)

Ecke (Vertex)
Zwei Koordinaten 0,
Einer ist 1

Baryzentrische Koordinaten (2D)

Bilineare Interpolation über ein Dreieck

$$V(\beta, \gamma) = (1 - \beta - \gamma)V_0 + \beta V_1 + \gamma V_2$$

Vertices

$$C(\beta, \gamma) = (1 - \beta - \gamma)C_0 + \beta C_1 + \gamma C_2$$

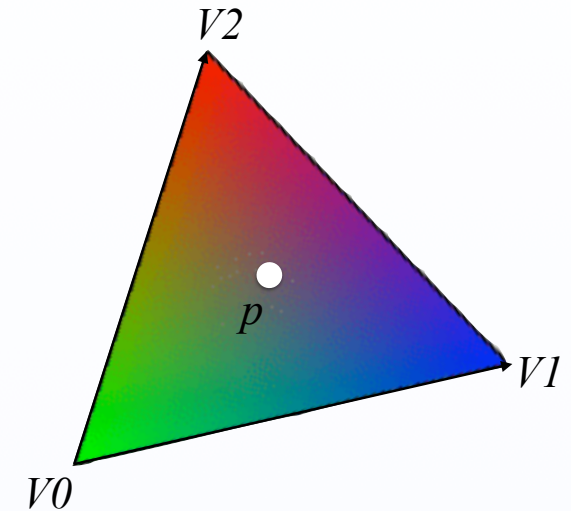
Colors (Gouraudshading)

$$V_n(\beta, \gamma) = (1 - \beta - \gamma)V_{n0} + \beta V_{n1} + \gamma V_{n2}$$

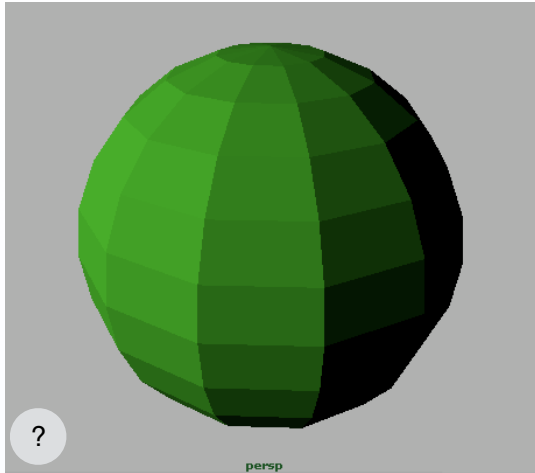
Normals (Phongshading)

$$V_t(\beta, \gamma) = (1 - \beta - \gamma)V_{t0} + \beta V_{t1} + \gamma V_{t2}$$

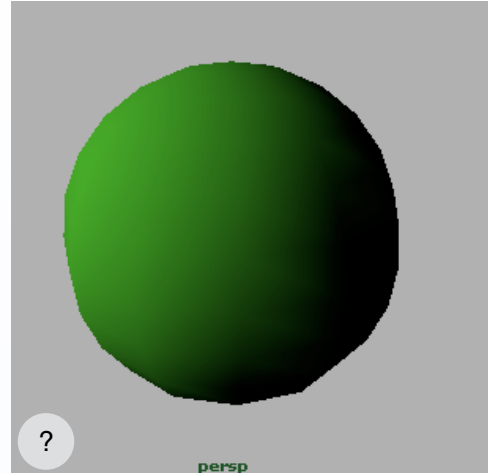
Texturecoordinates



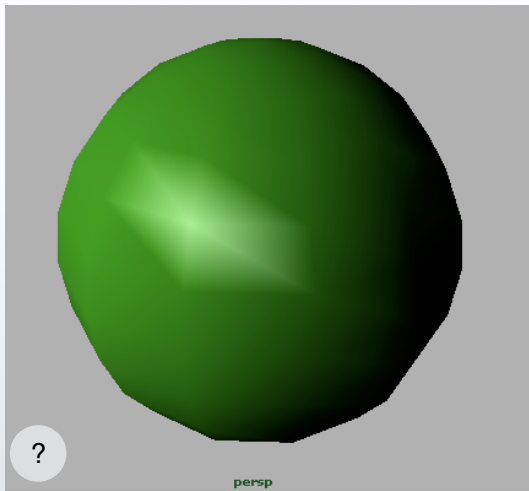
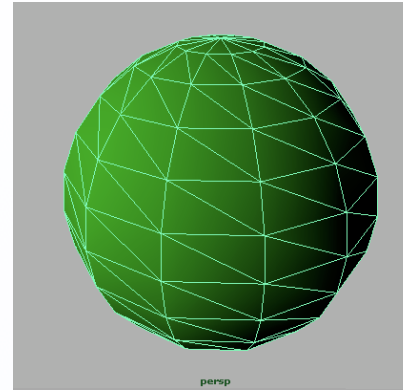
Illumination? Shading?



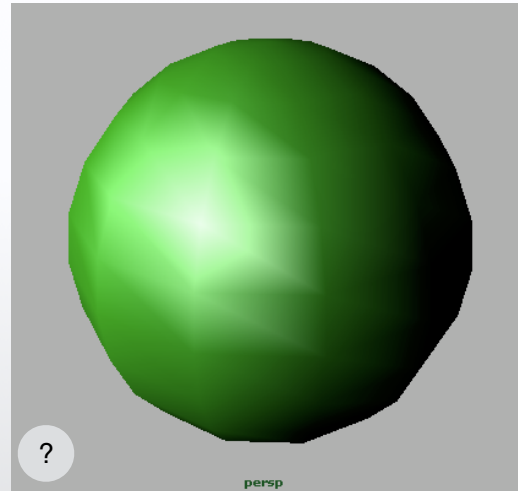
Flat-Lambert



Gouraud-Lambert

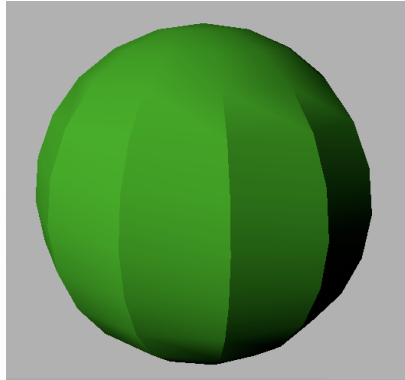


Gouraud-Phong



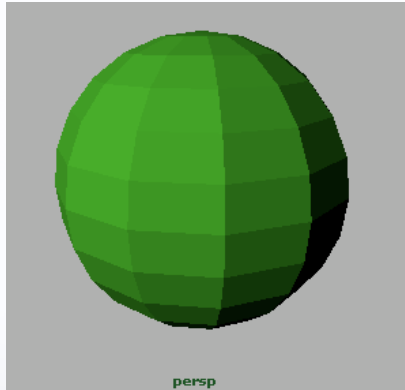
Gouraud-Blinn

Gouraud Shading • Nachteile



Machbandeffekt

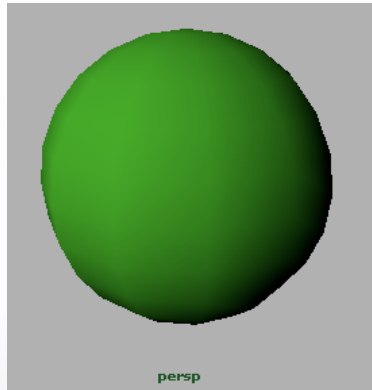
Das Auge verstärkt Kontrastverhältnisse zur besseren Kantenerkennung
Schlecht für Gouraud Shading



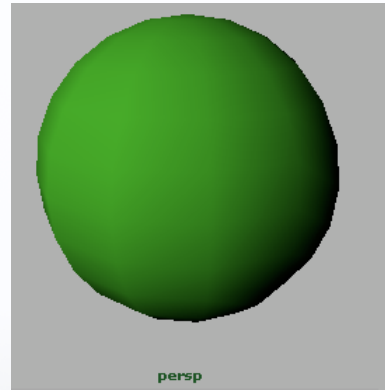
10°



20°



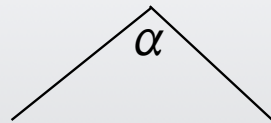
30°



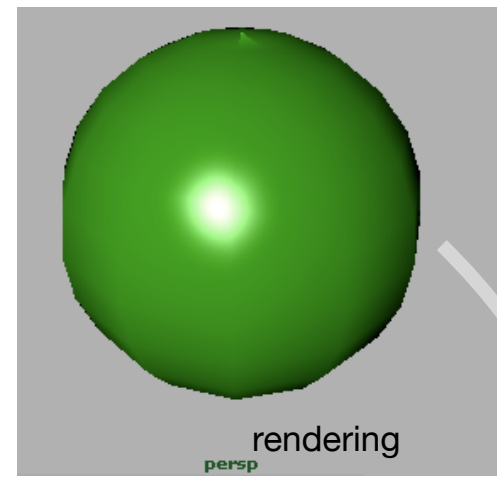
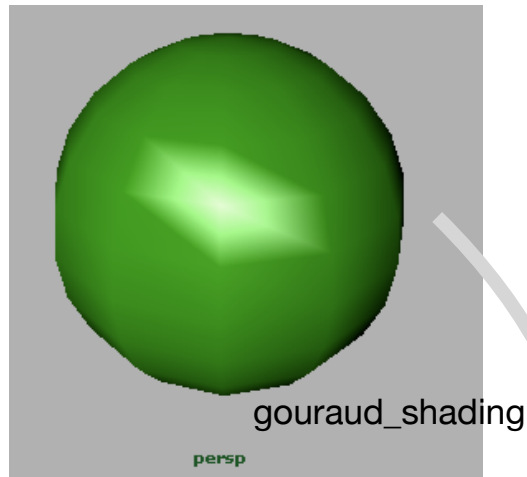
50°

Grenzwinkel ab dem nicht mehr interpoliert werden soll.

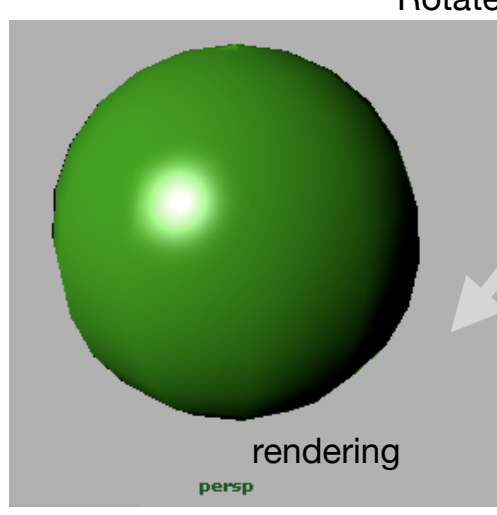
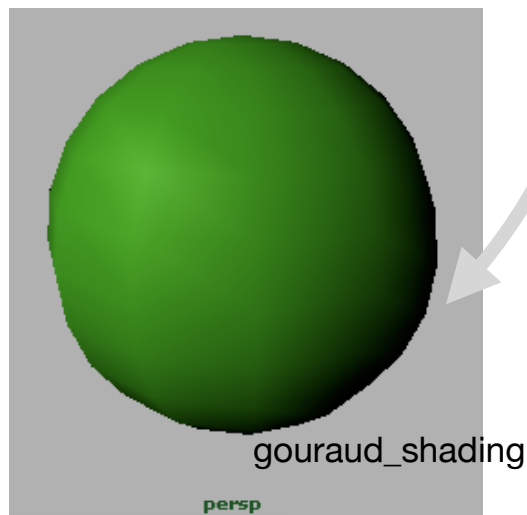
Gut bei gewollten Kanten
(Automobilindustrie)



Gouraud Shading • Nachteile

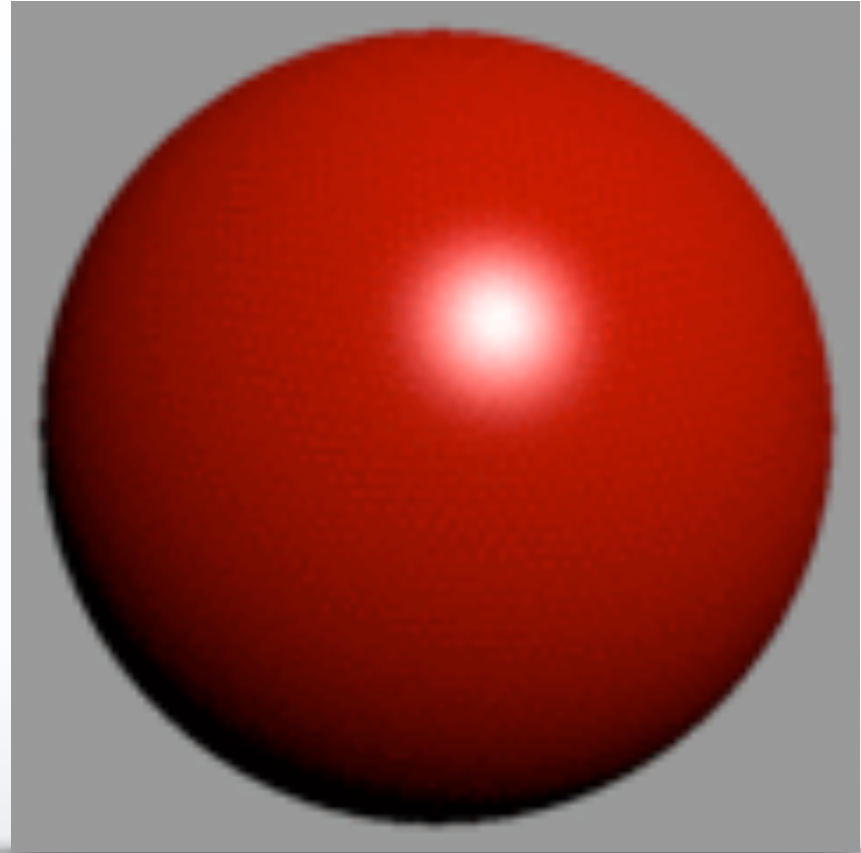
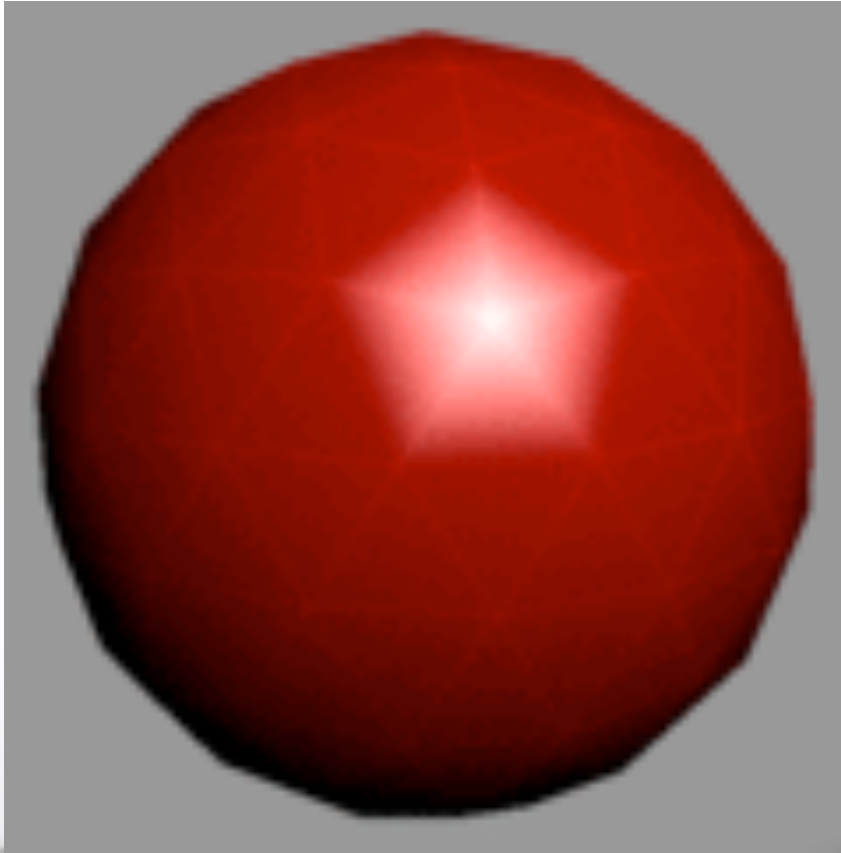


Rotate Y 30°

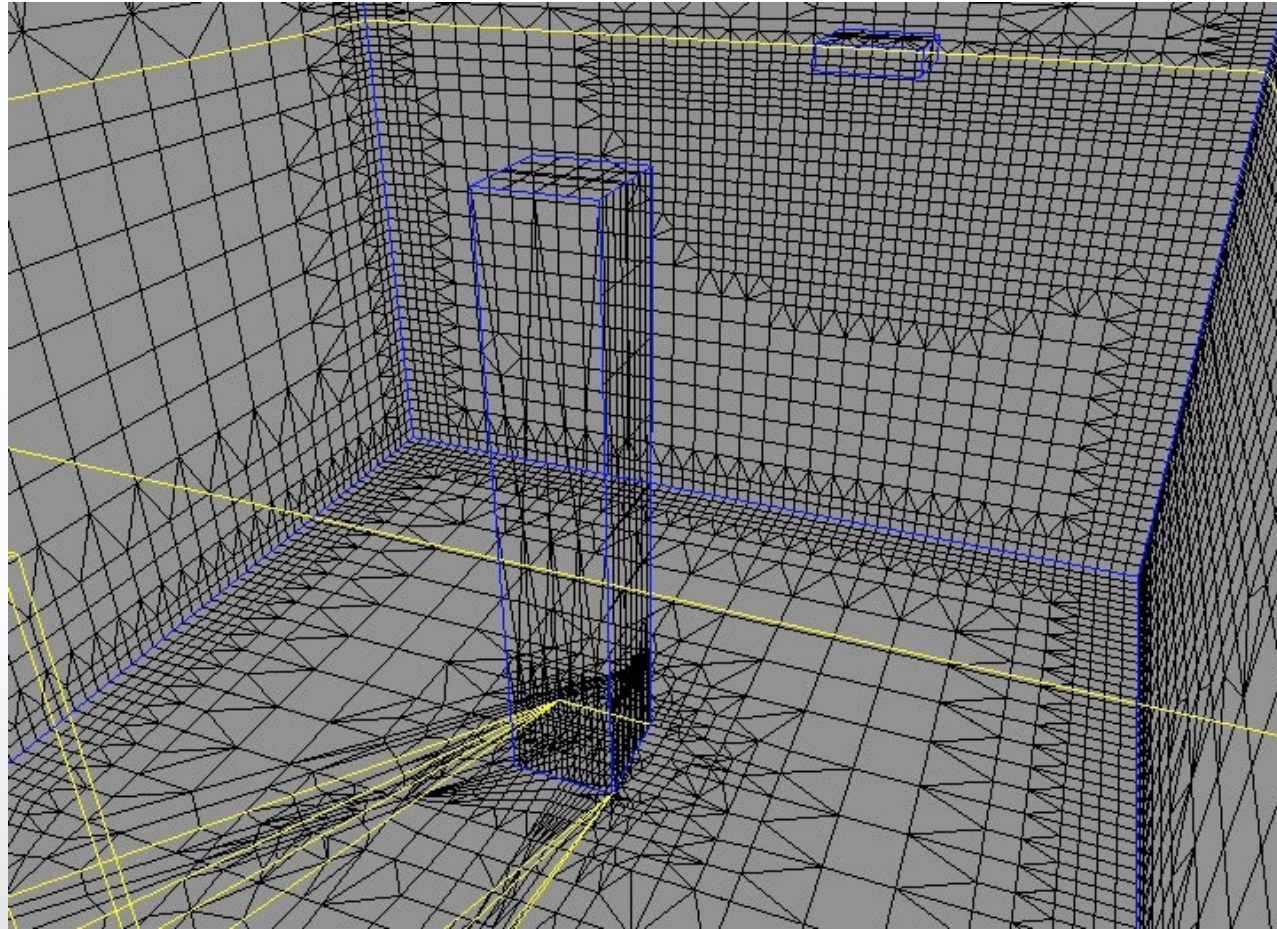


Rotate Y 30°

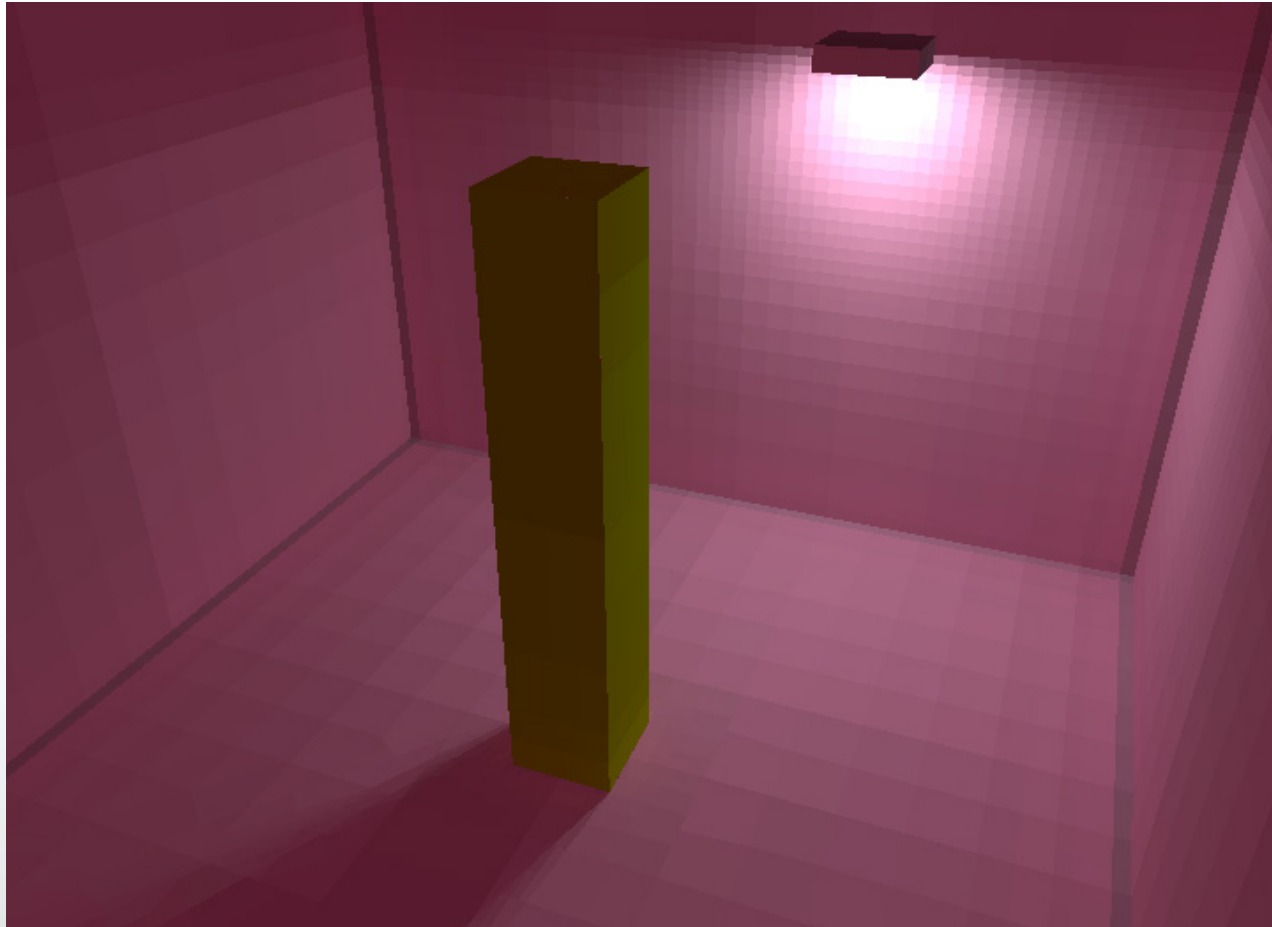
Gouraud Shading- Schwächen



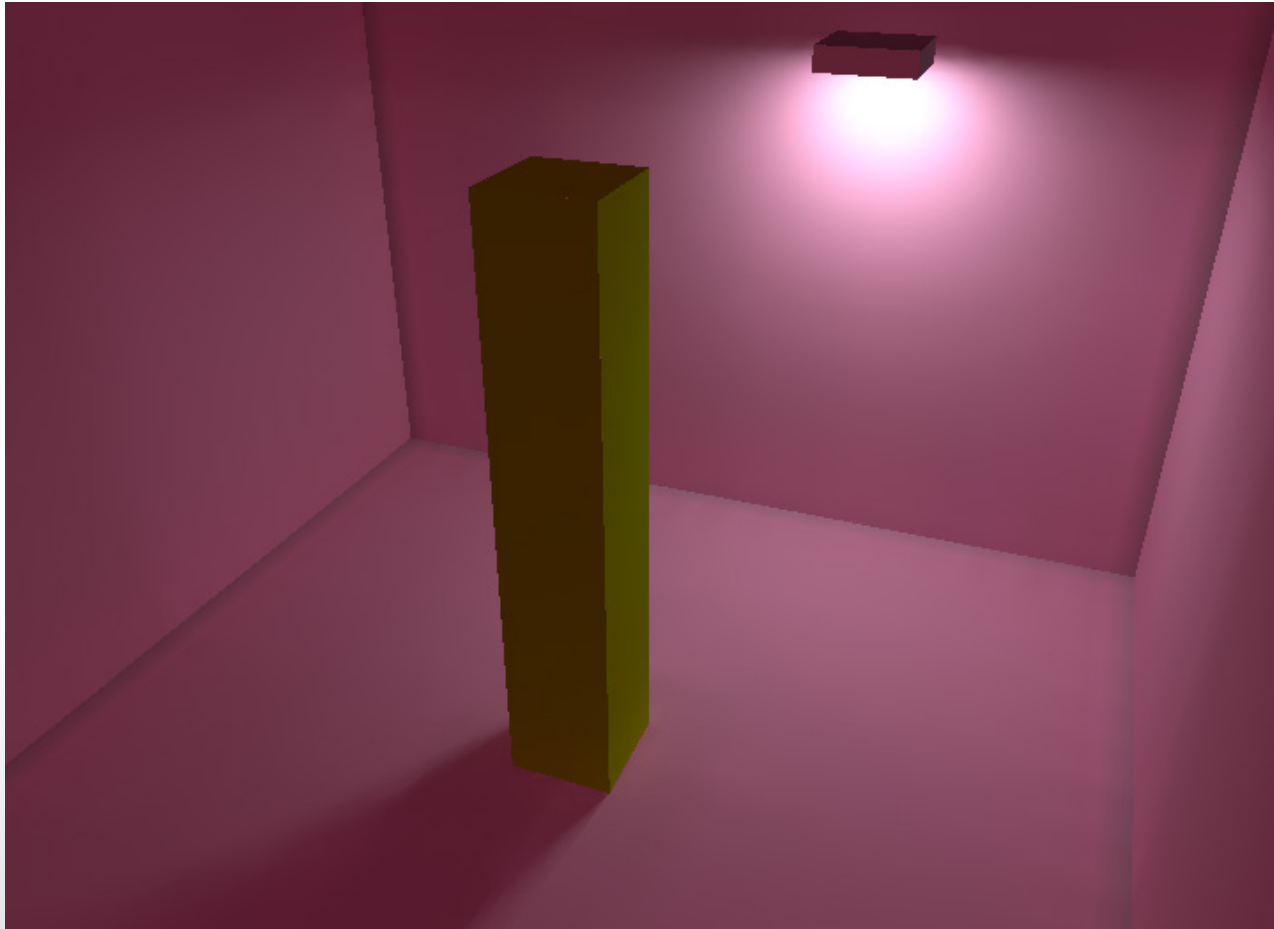
Wireframe



Flat shading



Gouraud Shading

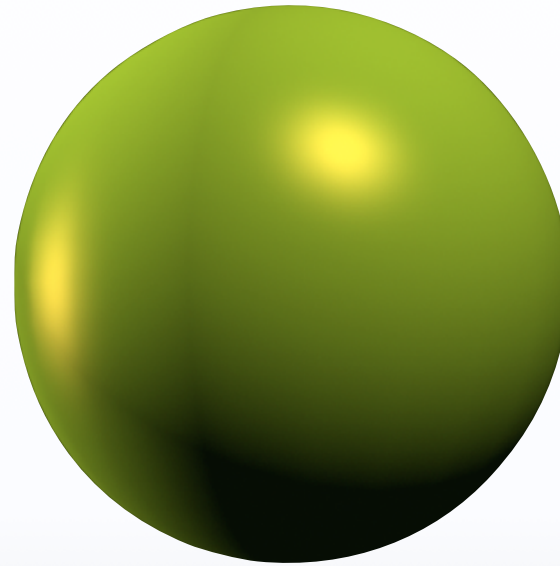




SMOOTH SHADING



Phong Shading (shader)



Phong Shading (rendering)

Phong Shading

